

CSC-3TC33

Algorithmique et fondements de l'informatique

Induction structurelle

Bertrand Meyer

Automne 2024

Des définitions qui se mordent la queue...

Exemple

La définition usuelle des entiers
est de dire :

Des définitions qui se mordent la queue...

Exemple

La définition usuelle des entiers est de dire :

« Un entier est soit l'élément 0,

Des définitions qui se mordent la queue...

Exemple

La définition usuelle des entiers est de dire :

« Un entier est soit l'élément 0,
soit le successeur d'un entier »

Des définitions qui se mordent la queue...

Exemple

La définition usuelle des entiers est de dire :

« Un entier est soit l'élément 0, soit le successeur d'un entier »

Exemple

Une manière de définir une liste est de dire :

« Une liste est soit la liste vide, soit le résultat de l'adjonction d'un élément en tête d'une liste. »



Des définitions qui se mordent la queue...

Exemple

Une manière de définir la fonction factorielle est de dire :

$$\text{fact}(n) = \begin{cases} 1 & \text{si } n = 0 \\ n \cdot \text{fact}(n - 1) & \text{sinon} \end{cases}$$



Des définitions qui se mordent la queue...

Exemple

Une manière de définir la fonction factorielle est de dire :

$$\text{fact}(n) = \begin{cases} 1 & \text{si } n = 0 \\ n \cdot \text{fact}(n - 1) & \text{sinon} \end{cases}$$

Objectif de cette séance

Clarifier ce mode de définition des objets.



Mots et langages

Alphabet

Définition

Un **alphabet** est un ensemble fini de symboles.



Alphabet

Définition

Un **alphabet** est un ensemble fini de symboles.

Exemple

$\Sigma = \{a, b\}$

Σ = ensemble des symboles de la table ASCII.



Les mots

Définition

Un **mot** sur l'alphabet Σ est une suite finie de symbole de Σ .

L'ensemble des mots se note Σ^* .

Exemple

Le mot **abbab** est un mot sur l'alphabet $\Sigma = \{a, b\}$.



Les mots

Définition

Un **mot** sur l'alphabet Σ est une suite finie de symbole de Σ .

L'ensemble des mots se note Σ^* .

Exemple

Le mot **abbab** est un mot sur l'alphabet $\Sigma = \{a, b\}$.



Définition

La longueur d'un **mot** est son nombre de lettre.

Exemple

Le mot **abbab** est de longueur 5.

Les mots

Définition

Un **mot** sur l'alphabet Σ est une suite finie de symbole de Σ .

L'ensemble des mots se note Σ^* .

Exemple

Le mot **abbab** est un mot sur l'alphabet $\Sigma = \{a, b\}$.



Définition

La longueur d'un **mot** est son nombre de lettre.

Exemple

Le mot **abbab** est de longueur 5.

Le mot vide se note ε : il est de longueur nulle.

Les opérations sur les mots

Définition

Si $u = x_1 \dots x_m \in \Sigma^*$ et $v = y_1 \dots y_n \in \Sigma^*$ sont deux mots, la **concaténation** de u et v est le mot

$$uv = x_1 \dots x_m y_1 \dots y_n \in \Sigma^*.$$



Les opérations sur les mots

Définition

Si $u = x_1 \dots x_m \in \Sigma^*$ et $v = y_1 \dots y_n \in \Sigma^*$ sont deux mots, la **concaténation** de u et v est le mot

$$uv = x_1 \dots x_m y_1 \dots y_n \in \Sigma^*.$$

Lorsque $w \in \Sigma^*$ et $r \in \mathbb{N}$, w^r désigne

$$\underbrace{w w \dots w}_{r \text{ fois}}.$$



Définition

Un **langage** est un ensemble de mots sur un alphabet, c'est-à-dire une partie de Σ^* .



Parler de syntaxe plutôt que des objets

Remarque pour les puristes

Pour éviter des paradoxes liés à des définitions d'ensembles, nous allons définir des objets comme sous-ensembles d'ensembles déjà connus.

En particulier, il est commode de définir des objets (par ex : listes, arbres) via la **syntaxe** qui les dénote, autrement dit comme mots sur un langage.

Définitions inductives

Définition

Soit n un entier naturel. On appelle **règle d'inférence** la donnée de $n + 1$ prédicats logiques : $P_1, \dots, P_n$ et Q . Les prédicats $P_1, \dots, P_n$ s'appellent les **prémisses**; le prédicat Q s'appelle la **conclusion**.

Définition

Soit n un entier naturel. On appelle **règle d'inférence** la donnée de $n + 1$ prédicats logiques : $P_1, \dots, P_n$ et Q . Les prédicats $P_1, \dots, P_n$ s'appellent les **prémisses**; le prédicat Q s'appelle la **conclusion**.

Cas particulier : lorsque $n = 0$, on parle d'**axiome**.

Définition

Soit n un entier naturel. On appelle **règle d'inférence** la donnée de $n + 1$ prédicats logiques : $P_1, \dots, P_n$ et Q . Les prédicats $P_1, \dots, P_n$ s'appellent les **prémisses**; le prédicat Q s'appelle la **conclusion**.

Cas particulier : lorsque $n = 0$, on parle d'**axiome**.

Définition

On parle de **système de règles d'inférence** pour désigner tout ensemble de règles d'inférences.

Notation d'une règle d'inférence

Prémisses : $P_1, \dots, P_n$.

Conclusion : Q .

On note

$$\frac{P_1 \quad P_2 \quad \dots \quad P_n}{Q} \text{ (nom de la règle)}$$

Définir un sous-ensemble par induction

Définition

Soit E un ensemble.

Une **définition inductive d'une partie $\mathcal{X}$ de E** est la donnée d'un système d'inférence dont chacune des règles est de la forme

$$\frac{x_1 \in \mathcal{X} \quad x_2 \in \mathcal{X} \quad \cdots \quad x_n \in \mathcal{X} \quad (x_1, \dots, x_n) \in \text{Dom}(r)}{r(x_1, \dots, x_n) \in \mathcal{X}} \quad (r)$$

pour une certaine fonction $r : E^n \rightarrow E$ dont le domaine de définition est noté $\text{Dom}(r)$.

Définir un sous-ensemble par induction

Théorème

Étant donné une définition inductive (D) d'une partie de E, il existe un ensemble $\mathcal{X}_0$ qui est le plus petit (au sens de l'inclusion) des ensembles $\mathcal{X}$ tels que, pour toute règle de (D), si $x_1 \in \mathcal{X}$, $x_2 \in \mathcal{X}$, ..., $x_n \in \mathcal{X}$ et si $r(x_1, x_2, \dots, x_n)$ est défini, alors,

$$r(x_1, x_2, \dots, x_n) \in \mathcal{X}.$$

Définir un sous-ensemble par induction

Théorème

Étant donné une définition inductive (D) d'une partie de E, il existe un ensemble $\mathcal{X}_0$ qui est le plus petit (au sens de l'inclusion) des ensembles $\mathcal{X}$ tels que, pour toute règle de (D), si $x_1 \in \mathcal{X}$, $x_2 \in \mathcal{X}$, ..., $x_n \in \mathcal{X}$ et si $r(x_1, x_2, \dots, x_n)$ est défini, alors,

$$r(x_1, x_2, \dots, x_n) \in \mathcal{X}.$$

Démonstration.

Soit $\mathcal{F}$ la famille de tels ensembles $\mathcal{X}$.

On pose

$$\mathcal{X}_0 = \bigcap_{\mathcal{X} \in \mathcal{F}} \mathcal{X}$$

et on prouve que $\mathcal{X}_0$ est minimal.



Exemple : définition inductive des entiers pairs

Exemple

La définition inductive d'une partie de $\mathbb{R}$ donnée par les deux règles d'inférence, dont un axiomes,

$$\frac{}{0 \in \mathcal{P}} \text{ (zero)}$$

$$\frac{n \in \mathcal{P}}{n + 2 \in \mathcal{P}} \text{ (successeur)}$$

permet de définir l'ensemble $\mathcal{P}$ des entiers pairs.

Exemple : définition inductive des entiers pairs

Exemple

La définition inductive d'une partie de $\mathbb{R}$ donnée par les deux règles d'inférence, dont un axiomes,

$$\frac{}{0 \in \mathcal{P}} \text{ (zero)}$$

$$\frac{n \in \mathcal{P}}{n + 2 \in \mathcal{P}} \text{ (successeur)}$$

permet de définir l'ensemble $\mathcal{P}$ des entiers pairs.

Remarque

Clairement, il existe de nombreuses parties $\mathcal{P}'$ de $\mathbb{R}$ qui vérifient

1. $0 \in \mathcal{P}'$
2. pour tout $n \in \mathbb{R}$, si $n \in \mathcal{P}'$ alors $n + 2 \in \mathcal{P}'$.

Par exemple $\mathcal{P}' = \{2n; n \in \mathbb{N}\}$, $\mathbb{N}$, $\mathbb{Z}$ ou $\mathbb{R}$. Mais $\mathcal{P} = \{2n; n \in \mathbb{N}\}$ est la plus petite d'entre elles pour $\subseteq$.

Exemple

Exemple

Soit $\Sigma = \{a, b, c\}$. La définition inductive d'une partie de Σ^* donnée par les deux règles d'inférence, dont un axiome,

$$\frac{}{b \in \mathcal{X}} \qquad \frac{w \in \mathcal{X}}{awc \in \mathcal{X}}$$

permet de définir l'ensemble $\mathcal{X}$ des mots de la forme

Exemple

Exemple

Soit $\Sigma = \{a, b, c\}$. La définition inductive d'une partie de Σ^* donnée par les deux règles d'inférence, dont un axiome,

$$\frac{}{b \in \mathcal{X}}$$

$$\frac{w \in \mathcal{X}}{awc \in \mathcal{X}}$$

permet de définir l'ensemble $\mathcal{X}$ des mots de la forme $a^n b c^n$ avec $n \in \mathbb{N}$:

$$\mathcal{X} = \{b, abc, aabcc, aaabccc, aaaabccccc, \dots\}.$$

Exemple : les palindromes

Exemple

Soit $\Sigma = \{a, b\}$. La définition inductive d'une partie de Σ^* donnée par les cinq règles d'inférence, dont trois axiomes,

$$\begin{array}{ccccc} \frac{}{\varepsilon \in \mathcal{P}} & \frac{}{a \in \mathcal{P}} & \frac{}{b \in \mathcal{P}} & \frac{w \in \mathcal{P}}{awa \in \mathcal{P}} & \frac{w \in \mathcal{P}}{bwb \in \mathcal{P}} \end{array}$$

permet de définir l'ensemble $\mathcal{P}$ des

Exemple : les palindromes

Exemple

Soit $\Sigma = \{a, b\}$. La définition inductive d'une partie de Σ^* donnée par les cinq règles d'inférence, dont trois axiomes,

$$\frac{}{\varepsilon \in \mathcal{P}} \quad \frac{}{a \in \mathcal{P}} \quad \frac{}{b \in \mathcal{P}} \quad \frac{w \in \mathcal{P}}{awa \in \mathcal{P}} \quad \frac{w \in \mathcal{P}}{bwb \in \mathcal{P}}$$

permet de définir l'ensemble $\mathcal{P}$ des palindromes.

Les listes

Définition inductive des listes

Définition

Soit E un ensemble.

L'ensemble $\mathcal{L}$ des listes à valeur dans E est la partie de Σ^* , où $\Sigma = E \cup \{\perp, (, ::,)\}$, définie inductivement par les règles d'inférence suivante :

$$\frac{}{\perp \in \mathcal{L}} \text{ (Liste vide)}$$

Définition inductive des listes

Définition

Soit E un ensemble.

L'ensemble $\mathcal{L}$ des listes à valeur dans E est la partie de Σ^* , où $\Sigma = E \cup \{\perp, (, ::,)\}$, définie inductivement par les règles d'inférence suivante :

$$\frac{}{\perp \in \mathcal{L}} \text{ (Liste vide)} \qquad \frac{q \in \mathcal{L}}{(e :: q) \in \mathcal{L}} \text{ (Liste de tête } e)$$

où il y a un axiome et autant de règles supplémentaires que d'éléments de E .

Définition inductive des listes

Définition

Soit E un ensemble.

L'ensemble $\mathcal{L}$ des listes à valeur dans E est la partie de Σ^* , où $\Sigma = E \cup \{\perp, (, ::,)\}$, définie inductivement par les règles d'inférence suivante :

$$\frac{}{\perp \in \mathcal{L}} \text{ (Liste vide)} \qquad \frac{q \in \mathcal{L}}{(e :: q) \in \mathcal{L}} \text{ (Liste de tête } e\text{)}$$

où il y a un axiome et autant de règles supplémentaires que d'éléments de E .

Traditionnellement, le symbole $::$ se lit « cons », le symbole $\perp$ se lit ici « nil ».

Définition inductive des listes

Définition

Soit E un ensemble.

L'ensemble $\mathcal{L}$ des listes à valeur dans E est la partie de Σ^* , où $\Sigma = E \cup \{\perp, (, ::,)\}$, définie inductivement par les règles d'inférence suivante :

$$\frac{}{\perp \in \mathcal{L}} \text{ (Liste vide)} \qquad \frac{q \in \mathcal{L}}{(e :: q) \in \mathcal{L}} \text{ (Liste de tête } e)$$

où il y a un axiome et autant de règles supplémentaires que d'éléments de E .

Traditionnellement, le symbole $::$ se lit « cons », le symbole $\perp$ se lit ici « nil ».

Exemple

La liste formée des entiers « 7 puis 9 puis 4 » correspond à

$$(7 :: (9 :: (4 :: \perp)))$$

Définition inductive des listes vs listes chaînées

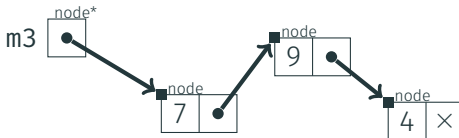
Définition inductive des listes $\leftrightarrow$ Listes chaînées

Exemple

La liste

`liste = (7::(9::(4::⊥)))`

correspond à la construction en mémoire (cf. INF107)



où l'on a déclaré une forme de maillons



Listes : correspondance règles & constructeurs

Correspondance

Règle d'inférence	Construction informatique
$\frac{}{\ell = \perp \in \mathcal{L}} \text{ (Liste vide)}$	
$\frac{q \in \mathcal{L}}{\ell = (e :: q) \in \mathcal{L}} \text{ (Liste de tête } e\text{)}$	

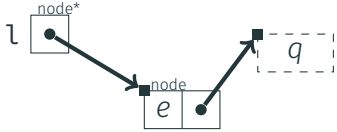
Listes : correspondance règles & constructeurs

Correspondance

Règle d'inférence	Construction informatique
$\frac{}{\ell = \perp \in \mathcal{L}} \text{ (Liste vide)}$	$\mathfrak{l} \begin{array}{ c } \text{node*} \\ \hline \times \end{array}$
$\frac{q \in \mathcal{L}}{\ell = (e :: q) \in \mathcal{L}} \text{ (Liste de tête e)}$	

Listes : correspondance règles & constructeurs

Correspondance

Règle d'inférence	Construction informatique
$\frac{}{\ell = \perp \in \mathcal{L}} \text{ (Liste vide)}$	$\mathcal{L} \quad \boxed{\times}^{\text{node}^*}$
$\frac{q \in \mathcal{L}}{\ell = (e :: q) \in \mathcal{L}} \text{ (Liste de tête } e \text{)}$	

Listes : correspondance règles & constructeurs en C

```
typedef struct node node_t;  
struct node {  
    int data;  
    node_t *next;  
};
```

Correspondance

Règle d'inférence	Construction informatique
$\frac{}{l = \perp \in \mathcal{L}} \text{ (Liste vide)}$	<code>l = NULL;</code>
$\frac{q \in \mathcal{L}}{l = (e :: q) \in \mathcal{L}} \text{ (Liste de tête } e \text{)}$	<pre>node_t *l = malloc(sizeof(node_t)); l->data = e; l->next = q;</pre>

Interlude : encapsulation de donnée en Python

En **programmation orientée objet**, on peut définir des **objets**, briques de base encapsulant des données et fonctions s'y rattachant. On parle :

- des **attributs** : données qui précisent les caractéristiques de l'objet.
- des **méthodes** : fonctions qui permettent d'interagir avec l'objet et qui en précisent le comportement.

Interlude : encapsulation de donnée en Python

```
class Rectangle():
    # on utilise self en argument 0 pour parler de l'objet lui-même
    # on peut donner des arguments par défaut
    # le constructeur est toujours donné par __init__
    def __init__(self, a, b, c = "no-name"):
        self.length = a
        self.width = b
        self.name = c
    # __str__ alimente la fonction print
    def __str__(self):
        return("I'm "+self.name)
    # self désigne l'objet
    def area(self):
        return self.length * self.width

r = Rectangle(3,4)
r.area() # notation pointée équivalant area(r)
print(r)
```

Listes : correspondance règles & constructeurs en Python

```
class Node:
    def __init__(self, dt, nxt = None):
        self.data = dt
        self.next = nxt
```

Correspondance

Règle d'inférence	Construction informatique
$\frac{}{\ell = \perp \in \mathcal{L}} \text{ (Liste vide)}$	$\ell = \text{None};$
$\frac{q \in \mathcal{L}}{\ell = (e :: q) \in \mathcal{L}} \text{ (Liste de tête } e)$	$\ell = \text{Node}(e, q)$

Remarque

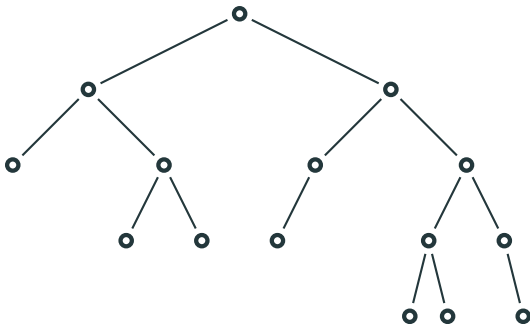
Il paraît évident que toute **liste inductive** est ainsi **représentable**. Un théorème le confirmera plus loin.

Les arbres

Les arbres binaires : introduction

Exemple

On voudrait représenter de l'information hiérarchique



Définition inductive des arbres binaires

Définition

L'ensemble $\mathcal{A}$ des arbres binaires est la partie de Σ^* , où $\Sigma = \{\perp, (, |,)\}$, définie inductivement par les règles d'inférence suivante :

Définition inductive des arbres binaires

Définition

L'ensemble $\mathcal{A}$ des arbres binaires est la partie de Σ^* , où $\Sigma = \{\perp, (, |,)\}$, définie inductivement par les règles d'inférence suivante :

$$\frac{}{\perp \in \mathcal{A}}$$

Définition inductive des arbres binaires

Définition

L'ensemble $\mathcal{A}$ des arbres binaires est la partie de Σ^* , où $\Sigma = \{\perp, (, |,)\}$, définie inductivement par les règles d'inférence suivante :

$$\frac{}{\perp \in \mathcal{A}}$$

$$\frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(g|d) \in \mathcal{A}}$$

Définition inductive des arbres binaires

Définition

L'ensemble $\mathcal{A}$ des arbres binaires est la partie de Σ^* , où $\Sigma = \{\perp, (, |,)\}$, définie inductivement par les règles d'inférence suivante :

$$\frac{}{\perp \in \mathcal{A}} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(g|d) \in \mathcal{A}}$$

Lorsqu'un arbre a est de la forme $a = (g|d)$, on dit que g est le sous-arbre *gauche* de l'arbre a et que d est le sous-arbre *droit* de l'arbre a .

Arbres : correspondance règles & constructions en Python (v1)

Python étant très tolérant, on peut construire des couples de n'importe quoi sans avoir à réfléchir à la gestion de la mémoire.

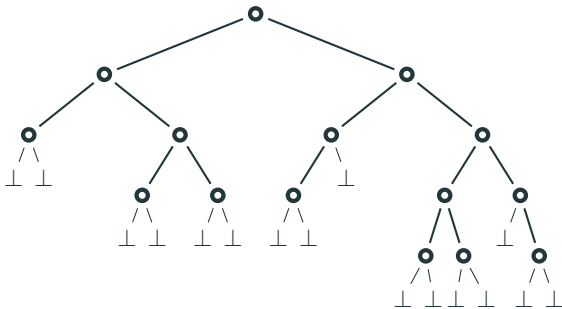
Correspondance

Règle d'inférence	Construction informatique
$\frac{}{a = \perp \in \mathcal{A}}$	<code>a = None;</code>
$\frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{a = (g \mid d) \in \mathcal{A}}$	<code>a = (g, d)</code>

Les arbres binaires : illustration (v1)

Exemple

L'arbre



peut s'écrire en Python

```
((None, None), ((None, None), (None, None)),  
((None, None), None),  
(((None, None), (None, None)), (None, (None, None))))
```

Pas très lisible !

Arbres : correspondance règles & constructions en C

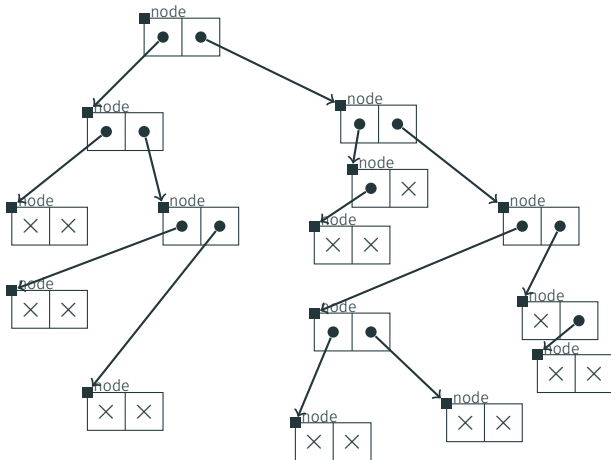
Rien de mieux que du C pour comprendre comment jouer avec la mémoire!

```
typedef struct node node_t;
struct node {
    node_t *left;
    node_t *right;
};
```

Correspondance

Règle d'inférence	Construction informatique
$\frac{}{a = \perp \in \mathcal{A}}$	<code>a = NULL;</code>
$\frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{a = (g d) \in \mathcal{A}}$	<pre>node_t *a = malloc(sizeof(node_t)); a->left = g; a->right = d;</pre>

Les arbres binaires : illustration (modèle C)



Arbres : correspondance règles & constructions en Python (v2)

```
class Node():  
    def __init__(self, a = None, b = None):  
        self.left = a  
        self.right = b
```

Correspondance

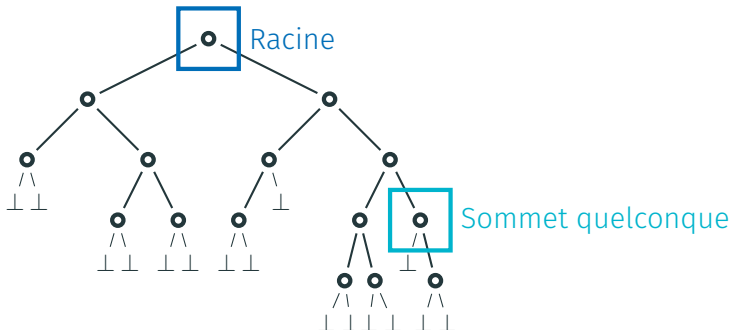
Règle d'inférence	Construction informatique
$\frac{}{a = \perp \in \mathcal{A}}$	a = None
$\frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{a = (g \mid d) \in \mathcal{A}}$	a = Node(g, d)

Racine et sommets des arbres

Définition

Soit $a = (g | d)$ un arbre : l'instance du constructeur $(\cdot | \cdot)$ employée est la **racine** de l'arbre. L'ensemble des racines des sous-arbres s'appellent les **nœuds** ou les **sommets** de l'arbre.

L'arbre vide n'a pas de racine.

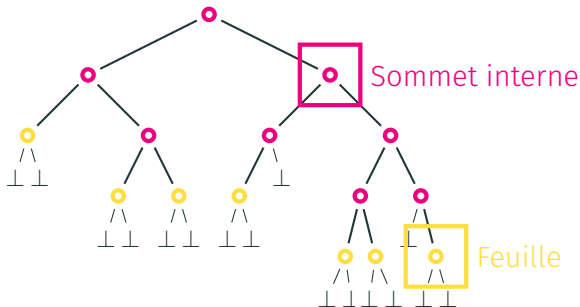


Nœuds interne et feuille des arbres

Définition

Un **sommet ou nœud interne** est un sommet possédant un ou deux sous-arbres non vide.

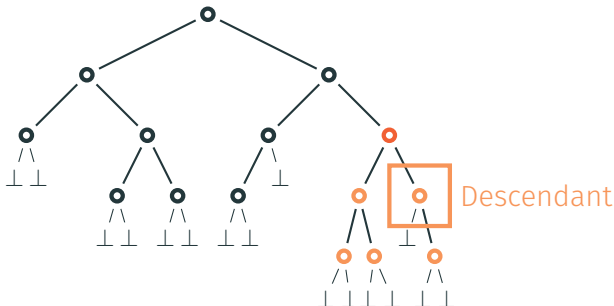
Une **feuille** est un sommet ayant ses deux sous-arbres vides.



Descendants d'un sommet

Définition

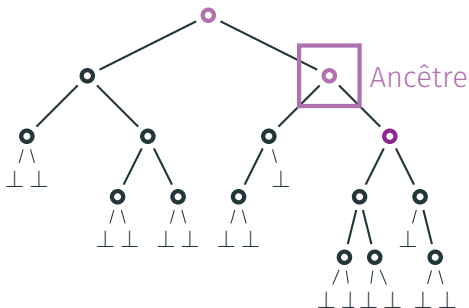
Un **descendant** d'un sommet s désigne tout sommet s' tel qu'il existe une suite de sommet $s = s_0, s_1, \dots, s_k = s'$ où pour tout $i \in \llbracket 0, k-1 \rrbracket$, le sommet s_{i+1} est un fils du sommet s_i .



Ancêtre d'un sommet

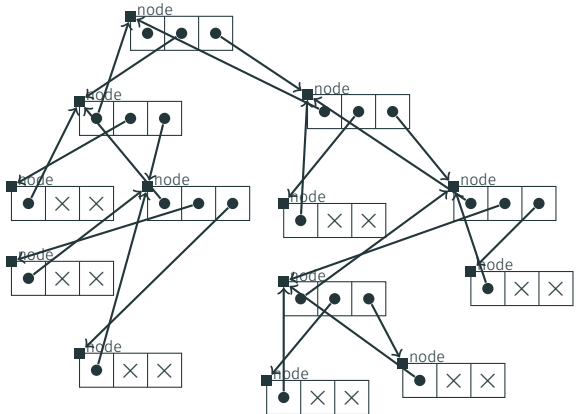
Définition

Un **ancêtre** d'un sommet s désigne tout sommet s' tel qu'il existe une suite de sommet $s = s_0, s_1, \dots, s_k = s'$ telle que pour tout indice $i \in \llbracket 0, k-1 \rrbracket$, le sommet s_{i+1} est le père du sommet s_i .



Constructions d'arbres binaires : variante avec le père

```
typedef struct node node_t;  
struct node {  
    node_t *parent;  
    node_t *left;  
    node_t *right;  
};
```



Arbres binaires complets

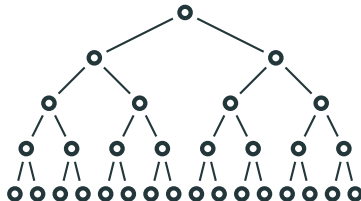
Définition

L'ensemble des arbres binaires complets est défini inductivement par

$$\frac{}{\perp \in \mathcal{C}}$$

$$\frac{c \in \mathcal{C}}{(c|c) \in \mathcal{C}}$$

Exemple



Définition inductive des arbres binaires étiquetés

Définition

Soit E un ensemble.

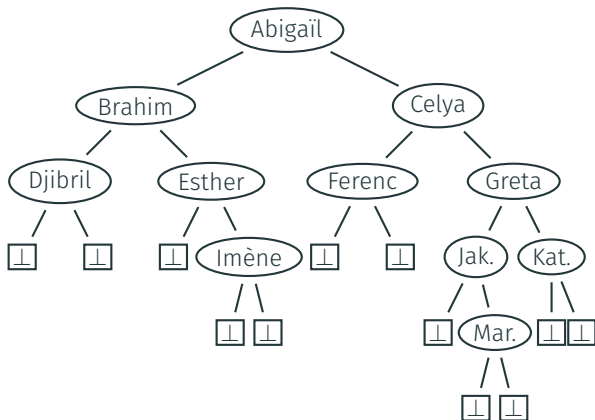
L'ensemble $\mathcal{A}$ des arbres binaire étiquetés dans E est la partie de Σ^* , où $\Sigma = E \cup \{\perp, (, |,)\}$, définie inductivement par les règles d'inférence suivante :

$$\frac{}{\perp \in \mathcal{A}} \text{ (Arbre vide)}$$

$$\frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(e|g|d) \in \mathcal{A}} \text{ (Arbre de racine étiquetée par } e\text{)}$$

où e appartient à E .

Illustration d'arbres binaires étiquetés



Adaptation des constructeurs

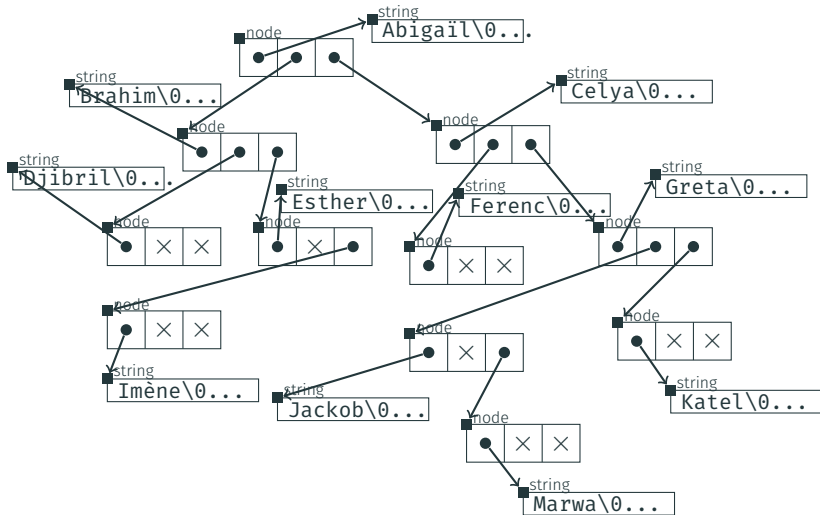
En langage C

```
typedef struct node node_t;  
struct node {  
    void *data;  
    node_t *left;  
    node_t *right;  
};
```

En langage Python

```
class Node():  
    def __init__(self, d, a = None, b = None):  
        self.data = d  
        self.left = a  
        self.right = b
```


Vision en mémoire des arbres binaires étiquetés



Définition inductive des arbres généraux étiquetés

Les arbres généraux non vides sont formés d'une racine et d'une liste d'arbres généraux non vides.

Définition

Soit E un ensemble.

L'ensemble $\mathcal{A}$ des arbres généraux étiquetés dans E non vide est la partie de Σ^* , où $\Sigma = E \cup \{\perp, (, |, ::,)\}$, définie inductivement par les règles d'inférence suivante :

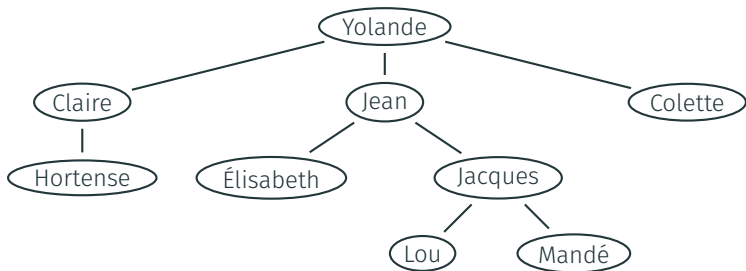
$$\frac{}{(e|\perp) \in \mathcal{A}} \text{ (Sing.)}$$

$$\frac{(e|l) \in \mathcal{A} \quad h \in \mathcal{A}}{(e|h::l) \in \mathcal{A}} \text{ (Ext)}$$

où e appartient à E .

Illustration d'arbre généraux étiquetés

Exemple



Arbres généraux : correspondance règles & constructions en C



```
typedef struct node node_t;  
struct node {  
    void *data;  
    node_t *first_child;  
    node_t *next_tree;  
};
```

Correspondance

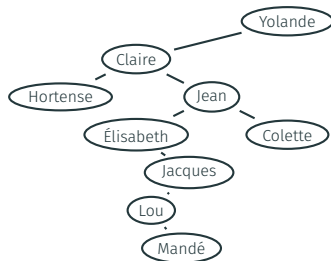
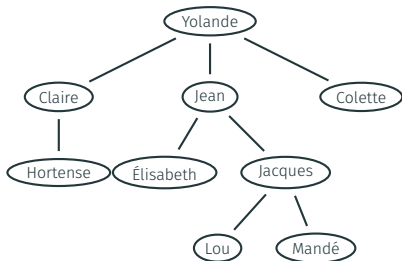
Règle d'inférence	Construction informatique
$\frac{}{a = (e \perp) \in \mathcal{A}}$	<pre>node_t *a = malloc(sizeof(node_t)); a->data = e; a->first_child = NULL; a->next_tree = NULL;</pre>
$\frac{a = (e \ell) \in \mathcal{A} \quad h \in \mathcal{A}}{a = (e h::\ell) \in \mathcal{A}}$	<pre>h->next_tree = a->first_child a->first_child = h;</pre>

Arbres généraux vs arbres binaires

La construction en C montre que les arbres généraux sont en **bijection** avec les arbres binaires non vides dont la racine n'a pas de fils droit.

Arbres généraux	Arbres binaires
Fils aîné Frère cadet	Fils gauche Fils droit

Exemple

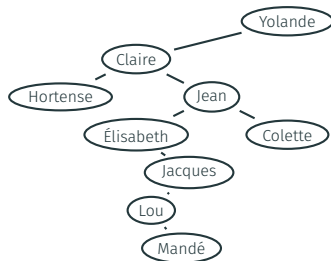
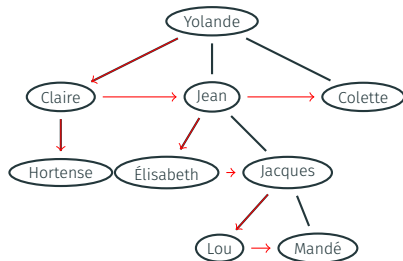


Arbres généraux vs arbres binaires

La construction en C montre que les arbres généraux sont en **bijection** avec les arbres binaires non vides dont la racine n'a pas de fils droit.

Arbres généraux	Arbres binaires
Fils aîné	Fils gauche
Frère cadet	Fils droit

Exemple



Arbres généraux : correspondance en Python

La manipulation d'arbres généraux est plus raisonnable en Python du fait de l'existence de listes natives en Python!

```
class Node():
    def __init__(self, d, a = None):
        self.data = d
        if a == None:
            self.children = [] # list of Nodes
        else:
            self.children = a
```

Correspondance

Règle d'inférence	Construction informatique
$\frac{}{a = (e \perp) \in \mathcal{A}}$	<code>a = Node(e)</code>
$\frac{a = (e \ell) \in \mathcal{A} \quad h \in \mathcal{A}}{a = (e h::\ell) \in \mathcal{A}}$	<code>a.children.append(h)</code>

Les expressions

Définition inductive des expressions arithmétiques

Définition

L'ensemble $\mathcal{E}$ des **expressions arithmétiques dans $\mathbb{N}$** est la partie de Σ^* , où $\Sigma = \mathbb{N} \cup \{ (, +, *,) \}$, définie inductivement par les règles d'inférence suivante :

$$\frac{}{n \in \mathcal{E}} \text{ (Entier } n)$$

$$\frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{(y+z) \in \mathcal{E}} \text{ (Somme)}$$

$$\frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{(y*z) \in \mathcal{E}} \text{ (Produit)}$$

où n appartient à $\mathbb{N}$.

NB : Nous pourrions rajouter les autres opérations usuelles !



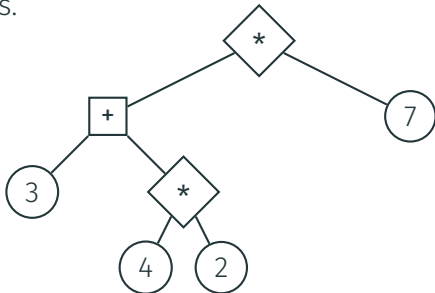
Expression arithmétique : illustration

Exemple

L'expression

$$x = ((3 + (4 * 2)) * 7)$$

correspond à un **arbre**, expliquant comment les trois types de règles ont été utilisés pour dériver x depuis les axiomes.



NB : nous continuons de parler d'arbre dans ce contexte !

Expressions arithmétique : règles & constructions

```
class BinaryOperator():
    def __init__(self, a, b):
        self.operand1 = a
        self.operand2 = b
# Astuce pour ne pas écrire __init__ deux fois :
# Sum et Product héritent du code de BinaryOperator
class Sum(BinaryOperator):
    pass
class Product(BinaryOperator):
    pass
```

Correspondance

Règle d'inférence	Construction informatique
$\frac{}{x = n \in \mathcal{E}} \text{ (Entier } n)$	$x = n$
$\frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{x = (y+z) \in \mathcal{E}} \text{ (Somme)}$	$x = \text{Sum}(y, z)$
$\frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{x = (y*z) \in \mathcal{E}} \text{ (Produit)}$	$x = \text{Product}(y, z)$

Définition inductive des formules de logique

Définition

Soient des **variables booléennes** $V = \{x_0, x_1, x_2, \dots\}$.

L'ensemble $\mathcal{F}$ des **formules de logique propositionnelle** est la partie de Σ^* , où $\Sigma = V \cup \{\neg, \wedge, \vee, (,)\}$, définie inductivement par les règles d'inférence suivante :

$$\frac{}{x_n \in \mathcal{F}} \text{ (Variable } n)$$

$$\frac{\varphi \in \mathcal{F}}{\neg\varphi \in \mathcal{F}} \text{ (Negation)}$$

$$\frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{(\varphi \vee \psi) \in \mathcal{F}} \text{ (Ou)}$$

$$\frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{(\varphi \wedge \psi) \in \mathcal{F}} \text{ (Et)}$$

où n appartient à $\mathbb{N}$.

NB : Nous pourrions rajouter les autres connecteurs usuels tels que $\rightarrow$ ou $\leftrightarrow$.

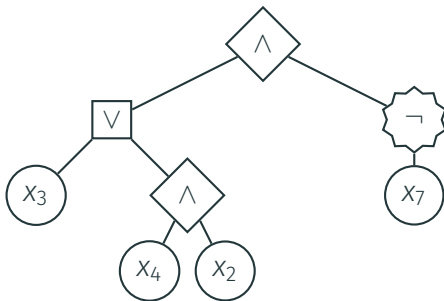
Formules logiques : illustration

Exemple

L'expression

$$y = ((x_3 \vee (x_4 \wedge x_2)) \wedge \neg x_7)$$

correspond à un **arbre**, expliquant comment les quatre types de règles ont été utilisés pour dériver y depuis les axiomes.



NB : nous continuons de parler d'arbre dans ce contexte !

Formules logiques : règles & constructions

```
class Neg():
    def __init__(self, a):
        self.operand = a
# Or et And héritent du code de BinaryOperator
class Or(BinaryOperator):
    pass
class And(BinaryOperator):
    pass
```

Correspondance

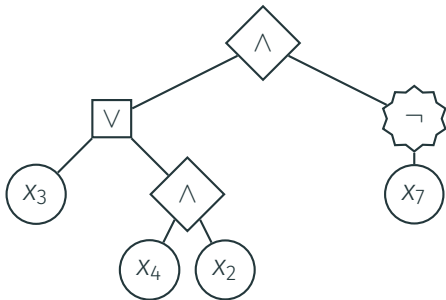
Règle d'inférence	Construction informatique
$\frac{}{z = x_n \in \mathcal{F}} \text{ (Variable } x_n)$	$z = n$
$\frac{\varphi \in \mathcal{F}}{z = \neg \varphi \in \mathcal{E}} \text{ (Negation)}$	$z = \text{Neg}(\text{phi})$
$\frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{z = (\varphi \vee \psi) \in \mathcal{F}} \text{ (Ou)}$	$z = \text{Or}(y, z)$
$\frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{z = (\varphi \wedge \psi) \in \mathcal{F}} \text{ (Et)}$	$z = \text{And}(y, z)$

Formules logiques : encodage

Exemple

L'expression

$$((x_3 \vee (x_4 \wedge x_2)) \wedge \neg x_7)$$



s'encode par

```
And(Or(3,And(4,2)),Neg(7))
```

Le raisonnement par induction structurelle

Principe d'induction structurelle

Théorème (Preuve par induction structurelle)

Soient $\mathcal{X}$ défini inductivement dans E par des règles

$$\frac{x_1 \in \mathcal{X} \quad x_2 \in \mathcal{X} \quad \dots \quad x_n \in \mathcal{X} \quad (x_1, \dots, x_n) \in \text{Dom}(r)}{r(x_1, \dots, x_n) \in \mathcal{X}} \quad (r)$$

et $\mathcal{P} : E \rightarrow \{\text{vrai}, \text{faux}\}$ un *prédicat*.

Alors

Principe d'induction structurelle

Théorème (Preuve par induction structurelle)

Soient $\mathcal{X}$ défini inductivement dans E par des règles

$$\frac{x_1 \in \mathcal{X} \quad x_2 \in \mathcal{X} \quad \dots \quad x_n \in \mathcal{X} \quad (x_1, \dots, x_n) \in \text{Dom}(r)}{r(x_1, \dots, x_n) \in \mathcal{X}} \quad (r)$$

et $\mathcal{P} : E \rightarrow \{\text{vrai}, \text{faux}\}$ un *prédicat*.

Alors

tout élément de $\mathcal{X}$ satisfait $\mathcal{P}$

si et seulement si,

Principe d'induction structurelle

Théorème (Preuve par induction structurelle)

Soient $\mathcal{X}$ défini inductivement dans E par des règles

$$\frac{x_1 \in \mathcal{X} \quad x_2 \in \mathcal{X} \quad \dots \quad x_n \in \mathcal{X} \quad (x_1, \dots, x_n) \in \text{Dom}(r)}{r(x_1, \dots, x_n) \in \mathcal{X}} \quad (r)$$

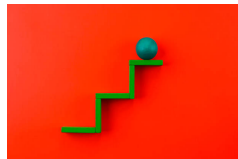
et $\mathcal{P} : E \rightarrow \{\text{vrai}, \text{faux}\}$ un *prédicat*.

Alors

tout élément de $\mathcal{X}$ satisfait $\mathcal{P}$

si et seulement si,

pour toute règle r , pour tout
 $(x_1, \dots, x_n) \in \text{Dom}(r)$,
 $\mathcal{P}(x_1), \mathcal{P}(x_2), \dots, \mathcal{P}(x_n)$ implique
 $\mathcal{P}(r(x_1, \dots, x_n))$.



Exemple de preuve par induction structurelle

Exemple

Soient $\Sigma = \{\mathbf{a}, \mathbf{b}\}$ et $\mathcal{L} \subseteq \Sigma^*$ le langage défini inductivement par

$$\frac{}{\varepsilon \in \mathcal{X}} \text{ (ax)}$$

$$\frac{w \in \mathcal{X}}{\mathbf{a}w\mathbf{b} \in \mathcal{X}} \text{ (r}_1\text{)}$$

$$\frac{w \in \mathcal{X}}{\mathbf{b}w\mathbf{a} \in \mathcal{X}} \text{ (r}_2\text{)}$$

et $\mathcal{P}(w) : \Sigma^* \rightarrow \{\mathbf{V}, \mathbf{F}\}$ le prédicat « il y a autant de **a** et **b** dans w ».

On veut montrer que $\mathcal{P}$ est vrai pour tout mot de $\mathcal{L}$.

Exemple de preuve par induction structurelle

Exemple

Soient $\Sigma = \{\mathbf{a}, \mathbf{b}\}$ et $\mathcal{L} \subseteq \Sigma^*$ le langage défini inductivement par

$$\frac{}{\varepsilon \in \mathcal{X}} \text{ (ax)}$$

$$\frac{w \in \mathcal{X}}{\mathbf{a}w\mathbf{b} \in \mathcal{X}} \text{ (r}_1\text{)}$$

$$\frac{w \in \mathcal{X}}{\mathbf{b}w\mathbf{a} \in \mathcal{X}} \text{ (r}_2\text{)}$$

et $\mathcal{P}(w) : \Sigma^* \rightarrow \{\mathbf{V}, \mathbf{F}\}$ le prédicat « il y a autant de $\mathbf{a}$ et $\mathbf{b}$ dans w ».

On veut montrer que $\mathcal{P}$ est vrai pour tout mot de $\mathcal{L}$.

Démonstration.

Cas de base : Si $w = \varepsilon$, alors $|w|_{\mathbf{a}} = 0$ et $|w|_{\mathbf{b}} = 0$. Donc $|w|_{\mathbf{a}} = |w|_{\mathbf{b}}$.

Exemple de preuve par induction structurelle

Exemple

Soient $\Sigma = \{\mathbf{a}, \mathbf{b}\}$ et $\mathcal{L} \subseteq \Sigma^*$ le langage défini inductivement par

$$\frac{}{\varepsilon \in \mathcal{X}} \text{ (ax)}$$

$$\frac{w \in \mathcal{X}}{\mathbf{a}w\mathbf{b} \in \mathcal{X}} \text{ (r}_1\text{)}$$

$$\frac{w \in \mathcal{X}}{\mathbf{b}w\mathbf{a} \in \mathcal{X}} \text{ (r}_2\text{)}$$

et $\mathcal{P}(w) : \Sigma^* \rightarrow \{\mathbf{V}, \mathbf{F}\}$ le prédicat « il y a autant de $\mathbf{a}$ et $\mathbf{b}$ dans w ».

On veut montrer que $\mathcal{P}$ est vrai pour tout mot de $\mathcal{L}$.

Démonstration.

Cas de base : Si $w = \varepsilon$, alors $|w|_{\mathbf{a}} = 0$ et $|w|_{\mathbf{b}} = 0$. Donc $|w|_{\mathbf{a}} = |w|_{\mathbf{b}}$.

Hérédité pour la règle (r_1) : Supposons que $|w|_{\mathbf{a}} = |w|_{\mathbf{b}}$ et intéressons nous au mot $w' = \mathbf{a}w\mathbf{b}$. On a $|w'|_{\mathbf{a}} = |w|_{\mathbf{a}} + 1$ et $|w'|_{\mathbf{b}} = |w|_{\mathbf{b}} + 1$. Mais alors $|w'|_{\mathbf{a}} = |w'|_{\mathbf{b}}$.

Exemple de preuve par induction structurelle

Exemple

Soient $\Sigma = \{a, b\}$ et $\mathcal{L} \subseteq \Sigma^*$ le langage défini inductivement par

$$\frac{}{\varepsilon \in \mathcal{X}} \text{ (ax)}$$

$$\frac{w \in \mathcal{X}}{awb \in \mathcal{X}} \text{ (r}_1\text{)}$$

$$\frac{w \in \mathcal{X}}{bwa \in \mathcal{X}} \text{ (r}_2\text{)}$$

et $\mathcal{P}(w) : \Sigma^* \rightarrow \{V, F\}$ le prédicat « il y a autant de **a** et **b** dans w ».

On veut montrer que $\mathcal{P}$ est vrai pour tout mot de $\mathcal{L}$.

Démonstration.

Cas de base : Si $w = \varepsilon$, alors $|w|_a = 0$ et $|w|_b = 0$. Donc $|w|_a = |w|_b$.

Hérédité pour la règle (r_1) : Supposons que $|w|_a = |w|_b$ et intéressons nous au mot $w' = awb$. On a $|w'|_a = |w|_a + 1$ et $|w'|_b = |w|_b + 1$. Mais alors $|w'|_a = |w'|_b$.

Hérédité pour la règle (r_2) : justification analogue.

□

Dérivations

Définition

On dit qu'une assertion est **dérivable** dans un système d'inférence s'il s'agit de la conclusion d'un axiome ou bien de la conclusion d'une règle dont les prémisses sont dérivables.

Définition

On dit qu'une assertion est **dérivable** dans un système d'inférence s'il s'agit de la conclusion d'un axiome ou bien de la conclusion d'une règle dont les prémisses sont dérivables.

Les éléments dérivables sont donc la réunion $\mathcal{D} = \bigcup_{m \in \mathbb{N}} \mathcal{D}_m$ où

$$\left\{ \begin{array}{l} \mathcal{D}_0 = \{\text{éléments en conclusion des axiomes}\} \\ \end{array} \right.$$

Définition

On dit qu'une assertion est **dérivable** dans un système d'inférence s'il s'agit de la conclusion d'un axiome ou bien de la conclusion d'une règle dont les prémisses sont dérivables.

Les éléments dérivables sont donc la réunion $\mathcal{D} = \bigcup_{m \in \mathbb{N}} \mathcal{D}_m$ où

$$\begin{cases} \mathcal{D}_0 &= \{\text{éléments en conclusion des axiomes}\} \\ \mathcal{D}_{m+1} &= \mathcal{D}_m \cup \bigcup_{r: \text{règle}} \{r(x_1, \dots, x_n) \in E; (x_1, \dots, x_n) \in (\mathcal{D}_m)^n \\ &\quad \text{et } (x_1, \dots, x_n) \in \text{Dom}(r)\} \end{cases}$$

Exemple de construction des éléments dérivables

Exemple

Soient $\Sigma = \{\mathbf{a}, \mathbf{b}\}$ et $\mathcal{L} \subseteq \Sigma^*$ le langage défini inductivement par

$$\frac{}{\varepsilon \in \mathcal{X}} \text{ (ax)}$$

$$\frac{w \in \mathcal{X}}{\mathbf{a}w\mathbf{b} \in \mathcal{X}} \text{ (r}_1\text{)}$$

$$\frac{w \in \mathcal{X}}{\mathbf{b}w\mathbf{a} \in \mathcal{X}} \text{ (r}_2\text{)}$$

On a

$$\mathcal{D}_0 =$$

Exemple de construction des éléments dérivables

Exemple

Soient $\Sigma = \{\mathbf{a}, \mathbf{b}\}$ et $\mathcal{L} \subseteq \Sigma^*$ le langage défini inductivement par

$$\frac{}{\varepsilon \in \mathcal{X}} \text{ (ax)}$$

$$\frac{w \in \mathcal{X}}{awb \in \mathcal{X}} \text{ (r}_1\text{)}$$

$$\frac{w \in \mathcal{X}}{bwa \in \mathcal{X}} \text{ (r}_2\text{)}$$

On a

$$\mathcal{D}_0 = \{\varepsilon\}$$

$$\mathcal{D}_1 =$$

Exemple de construction des éléments dérivables

Exemple

Soient $\Sigma = \{\mathbf{a}, \mathbf{b}\}$ et $\mathcal{L} \subseteq \Sigma^*$ le langage défini inductivement par

$$\frac{}{\varepsilon \in \mathcal{X}} \text{ (ax)}$$

$$\frac{w \in \mathcal{X}}{\mathbf{a}w\mathbf{b} \in \mathcal{X}} \text{ (r}_1\text{)}$$

$$\frac{w \in \mathcal{X}}{\mathbf{b}w\mathbf{a} \in \mathcal{X}} \text{ (r}_2\text{)}$$

On a

$$\mathcal{D}_0 =$$

$$\{\varepsilon\}$$

$$\mathcal{D}_1 =$$

$$\{\varepsilon, \mathbf{ab}, \mathbf{ba}\}$$

$$\mathcal{D}_2 =$$

Exemple de construction des éléments dérivables

Exemple

Soient $\Sigma = \{\mathbf{a}, \mathbf{b}\}$ et $\mathcal{L} \subseteq \Sigma^*$ le langage défini inductivement par

$$\frac{}{\varepsilon \in \mathcal{X}} \text{ (ax)}$$

$$\frac{w \in \mathcal{X}}{awb \in \mathcal{X}} \text{ (r}_1\text{)}$$

$$\frac{w \in \mathcal{X}}{bwa \in \mathcal{X}} \text{ (r}_2\text{)}$$

On a

$$\mathcal{D}_0 = \{\varepsilon\}$$

$$\mathcal{D}_1 = \{\varepsilon, \mathbf{ab}, \mathbf{ba}\}$$

$$\mathcal{D}_2 = \{\varepsilon, \mathbf{ab}, \mathbf{ba}, \mathbf{aabb}, \mathbf{abab}, \mathbf{baba}, \mathbf{bbaa}\}$$

$$\mathcal{D}_3 =$$

Exemple de construction des éléments dérivables

Exemple

Soient $\Sigma = \{\mathbf{a}, \mathbf{b}\}$ et $\mathcal{L} \subseteq \Sigma^*$ le langage défini inductivement par

$$\frac{}{\varepsilon \in \mathcal{X}} \text{ (ax)}$$

$$\frac{w \in \mathcal{X}}{awb \in \mathcal{X}} \text{ (r}_1\text{)}$$

$$\frac{w \in \mathcal{X}}{bwa \in \mathcal{X}} \text{ (r}_2\text{)}$$

On a

$$\mathcal{D}_0 = \{\varepsilon\}$$

$$\mathcal{D}_1 = \{\varepsilon, \mathbf{ab}, \mathbf{ba}\}$$

$$\mathcal{D}_2 = \{\varepsilon, \mathbf{ab}, \mathbf{ba}, \mathbf{aabb}, \mathbf{abab}, \mathbf{baba}, \mathbf{bbaa}\}$$

$$\mathcal{D}_3 = \{\varepsilon, \mathbf{ab}, \mathbf{ba}, \mathbf{aabb}, \mathbf{abab}, \mathbf{baba}, \mathbf{bbaa}, \\ \mathbf{aaabbb}, \mathbf{aababb}, \mathbf{ababab}, \mathbf{abbaab}, \\ \mathbf{baabba}, \mathbf{bababa}, \mathbf{bbabaa}, \mathbf{bbbbaa}\}$$

$$\mathcal{D}_4 = \dots$$

En fait, les mots dérivables sont les mots de Σ^* avec autant de **a** que de **b**.

Quand on peut dériver un élément x à partir des axiomes, on présente la dérivation sous forme d'**arbre**

- Les feuilles sont étiquetées par les axiomes;
- les sommets internes sont étiquetés par des instanciations des règles.
- L'élément x est dans la conclusion de la racine;
- les prémisses d'un sommet interne sont les conclusions de ses fils.

Illustration des arbres de dérivation

Exemple

L'expression

$$((x_3 \vee (x_4 \wedge x_2)) \wedge \neg x_7)$$

a pour arbre de dérivation

$$\frac{\frac{\frac{x_3 \in \mathcal{F}}{x_3 \in \mathcal{F}} \text{ (Var } x_3)}{\frac{\frac{\frac{x_4 \in \mathcal{F}}{x_4 \in \mathcal{F}} \text{ (Var } x_4)}{\frac{\frac{x_2 \in \mathcal{F}}{x_2 \in \mathcal{F}} \text{ (Var } x_2)}{(x_4 \wedge x_2) \in \mathcal{F}} \text{ (Et)}}{(x_3 \vee (x_4 \wedge x_2)) \in \mathcal{F}} \text{ (Ou)}}{((x_3 \vee (x_4 \wedge x_2)) \wedge \neg x_7) \in \mathcal{F}} \text{ (Et)} \quad \frac{\frac{x_7 \in \mathcal{F}}{\neg x_7 \in \mathcal{F}} \text{ (Nég)}}{\neg x_7 \in \mathcal{F}} \text{ (Var } x_7)$$

Illustration des arbres de dérivation

Exemple

L'expression

$$((x_3 \vee (x_4 \wedge x_2)) \wedge \neg x_7)$$

a pour arbre de dérivation

$$\frac{\frac{\frac{x_3 \in \mathcal{F}}{x_3 \in \mathcal{F}} \text{ (Var } x_3)}{\frac{\frac{\frac{x_4 \in \mathcal{F}}{x_4 \in \mathcal{F}} \text{ (Var } x_4)}{\frac{\frac{x_2 \in \mathcal{F}}{x_2 \in \mathcal{F}} \text{ (Var } x_2)}{(x_4 \wedge x_2) \in \mathcal{F}} \text{ (Et)}}{(x_3 \vee (x_4 \wedge x_2)) \in \mathcal{F}} \text{ (Ou)}}{((x_3 \vee (x_4 \wedge x_2)) \wedge \neg x_7) \in \mathcal{F}} \text{ (Et)} \quad \frac{\frac{x_7 \in \mathcal{F}}{\neg x_7 \in \mathcal{F}} \text{ (Nég)}}{\neg x_7 \in \mathcal{F}} \text{ (Var } x_7)$$

Théorème de représentation

Théorème

Soit $\mathcal{X}$ une partie d'un ensemble E définie inductivement, alors les éléments de $\mathcal{X}$ coïncident avec les éléments dérivables $\mathcal{D}$.

Théorème de représentation

Théorème

Soit $\mathcal{X}$ une partie d'un ensemble E définie inductivement, alors les éléments de $\mathcal{X}$ coïncident avec les éléments dérivables $\mathcal{D}$.

Démonstration.

Soit $\mathcal{P}(x)$ le prédicat $x \in \mathcal{D}$. Ce prédicat est vrai pour les axiomes (à cause de $\mathcal{D}_0$) et est stable pour chacune des règles de la définition de $\mathcal{X}$. D'après le théorème de preuve par induction structurelle, $\mathcal{P}(x)$ est vrai pour tout élément de $\mathcal{X}$. Autrement dit $\mathcal{X} \subseteq \mathcal{D}$.

Théorème de représentation

Théorème

Soit $\mathcal{X}$ une partie d'un ensemble E définie inductivement, alors les éléments de $\mathcal{X}$ coïncident avec les éléments dérivables $\mathcal{D}$.

Démonstration.

Soit $\mathcal{P}(x)$ le prédicat $x \in \mathcal{D}$. Ce prédicat est vrai pour les axiomes (à cause de $\mathcal{D}_0$) et est stable pour chacune des règles de la définition de $\mathcal{X}$. D'après le théorème de preuve par induction structurelle, $\mathcal{P}(x)$ est vrai pour tout élément de $\mathcal{X}$. Autrement dit $\mathcal{X} \subseteq \mathcal{D}$.

Pour l'inclusion inverse, nous raisonnons par récurrence.

On doit avoir $\mathcal{D}_0 \subseteq \mathcal{X}$ car il s'agit des axiomes.

Par ailleurs, supposons que $\mathcal{D}_m \subseteq \mathcal{X}$. Un élément y de $\mathcal{D}_{m+1}$ est soit dans $\mathcal{D}_m$, soit de la forme $r(x_1, \dots, x_n)$ où $(x_1, \dots, x_n) \in \mathcal{D}_m \subseteq \mathcal{X}$. Mais comme $\mathcal{X}$ est stable pour la règle r , $y \in \mathcal{X}$. Ainsi, dans les deux cas, y appartient à $\mathcal{D}_{m+1}$ et on a $\mathcal{D}_{m+1} \subseteq \mathcal{X}$. Mais alors, on a aussi

$$\mathcal{D} = \bigcup_{m \in \mathbb{N}} \mathcal{D}_m \subseteq \mathcal{X}.$$

□

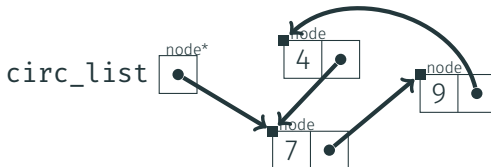
Représentation et bilan

Remarque

Nous sommes rassurés par le théorème de représentation : il prouve que tous nos objets définis par **induction structurelle**, tels que les listes, arbres binaires, formule sont bien **représentables informatiquement** par les constructeurs.

Co-remarque

L'inverse n'est pas vrai, nous pouvons même représenter plus de choses : par exemple des listes circulaires.



Fonctions définies par induction structurelle

Définition

On dit que la définition par induction d'un sous-ensemble $\mathcal{X}$ est **non ambiguë** dès lors qu'il n'existe qu'une seule manière de produire chaque élément de X :

Autrement dit, pour tout $x \in \mathcal{X}$, il n'existe, qu'un seul arbre de dérivation.

Théorème

Soient E et F deux ensembles et $\mathcal{X}$ un sous-ensemble de E défini par une définition inductive D non ambiguë.

Théorème

Soient E et F deux ensembles et $\mathcal{X}$ un sous-ensemble de E défini par une définition inductive D non ambiguë.

Soit, pour chacune des règles $r : E^{n_r} \rightarrow E$ de la définition D , une application $g_r : E^{n_r} \times F^{n_r} \rightarrow F$.

Construction de fonctions récursives

Théorème

Soient E et F deux ensembles et $\mathcal{X}$ un sous-ensemble de E défini par une définition inductive D non ambiguë.

Soit, pour chacune des règles $r : E^{n_r} \rightarrow E$ de la définition D , une application $g_r : E^{n_r} \times F^{n_r} \rightarrow F$.

Alors il existe une unique application $f : \mathcal{X} \rightarrow F$ telle que, pour toute règle r ,

$$\forall (x_1, \dots, x_n) \in \mathcal{X}^{n_r} \cap \text{Dom}(r),$$

$$f(r(x_1, \dots, x_{n_r})) = g_r(x_1, \dots, x_{n_r}, f(x_1), \dots, f(x_{n_r}))$$

Construction de fonctions récursives

Théorème

Soient E et F deux ensembles et $\mathcal{X}$ un sous-ensemble de E défini par une définition inductive D non ambiguë.

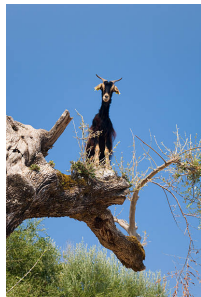
Soit, pour chacune des règles $r : E^{n_r} \rightarrow E$ de la définition D , une application $g_r : E^{n_r} \times F^{n_r} \rightarrow F$.

Alors il existe une unique application $f : \mathcal{X} \rightarrow F$ telle que, pour toute règle r ,

$$\forall (x_1, \dots, x_n) \in \mathcal{X}^{n_r} \cap \text{Dom}(r),$$

$$f(r(x_1, \dots, x_{n_r})) = g_r(x_1, \dots, x_{n_r}, f(x_1), \dots, f(x_{n_r}))$$

Autrement dit, on peut programmer récursivement sans crainte.



Construction de fonctions récursives : démonstration

Démonstration.

Soient $f_1 : \mathcal{X} \rightarrow F$ et $f_2 : \mathcal{X} \rightarrow F$ deux applications répondant au théorème. Nous introduisons le prédicat $\mathcal{P}(x) : \ll f_1(x) = f_2(x) \gg$.

Construction de fonctions récursives : démonstration

Démonstration.

Soient $f_1 : \mathcal{X} \rightarrow F$ et $f_2 : \mathcal{X} \rightarrow F$ deux applications répondant au théorème. Nous introduisons le prédicat $\mathcal{P}(x) : \ll f_1(x) = f_2(x) \gg$. Soit r une règle,

Construction de fonctions récursives : démonstration

Démonstration.

Soient $f_1 : \mathcal{X} \rightarrow F$ et $f_2 : \mathcal{X} \rightarrow F$ deux applications répondant au théorème. Nous introduisons le prédicat $\mathcal{P}(x) : \ll f_1(x) = f_2(x) \gg$. Soit r une règle, donnons nous $(x_1, \dots, x_n) \in \mathcal{X}^{n_r} \cap \text{Dom}(r)$. Supposons $\mathcal{P}(x_1), \dots, \mathcal{P}(x_n)$. Alors

Construction de fonctions récursives : démonstration

Démonstration.

Soient $f_1 : \mathcal{X} \rightarrow F$ et $f_2 : \mathcal{X} \rightarrow F$ deux applications répondant au théorème. Nous introduisons le prédicat $\mathcal{P}(x) : \ll f_1(x) = f_2(x) \gg$. Soit r une règle, donnons nous $(x_1, \dots, x_n) \in \mathcal{X}^{n_r} \cap \text{Dom}(r)$. Supposons $\mathcal{P}(x_1), \dots, \mathcal{P}(x_n)$. Alors

$$f_1(r(x_1, \dots, x_{n_r})) = g_r(x_1, \dots, x_{n_r}, f_1(x_1), \dots, f_1(x_{n_r}))$$

Construction de fonctions récursives : démonstration

Démonstration.

Soient $f_1 : \mathcal{X} \rightarrow F$ et $f_2 : \mathcal{X} \rightarrow F$ deux applications répondant au théorème. Nous introduisons le prédicat $\mathcal{P}(x) : \ll f_1(x) = f_2(x) \gg$. Soit r une règle, donnons nous $(x_1, \dots, x_n) \in \mathcal{X}^{n_r} \cap \text{Dom}(r)$. Supposons $\mathcal{P}(x_1), \dots, \mathcal{P}(x_n)$. Alors

$$\begin{aligned} f_1(r(x_1, \dots, x_{n_r})) &= g_r(x_1, \dots, x_{n_r}, f_1(x_1), \dots, f_1(x_{n_r})) \\ &= g_r(x_1, \dots, x_{n_r}, f_2(x_1), \dots, f_2(x_{n_r})) \end{aligned}$$

Construction de fonctions récursives : démonstration

Démonstration.

Soient $f_1 : \mathcal{X} \rightarrow F$ et $f_2 : \mathcal{X} \rightarrow F$ deux applications répondant au théorème. Nous introduisons le prédicat $\mathcal{P}(x) : \ll f_1(x) = f_2(x) \gg$. Soit r une règle, donnons nous $(x_1, \dots, x_n) \in \mathcal{X}^{n_r} \cap \text{Dom}(r)$. Supposons $\mathcal{P}(x_1), \dots, \mathcal{P}(x_n)$. Alors

$$\begin{aligned} f_1(r(x_1, \dots, x_{n_r})) &= g_r(x_1, \dots, x_{n_r}, f_1(x_1), \dots, f_1(x_{n_r})) \\ &= g_r(x_1, \dots, x_{n_r}, f_2(x_1), \dots, f_2(x_{n_r})) \\ &= f_2(r(x_1, \dots, x_{n_r})) \end{aligned}$$

Mais alors on a $\mathcal{P}(r(x_1, \dots, x_{n_r}))$.

Construction de fonctions récursives : démonstration

Démonstration.

Soient $f_1 : \mathcal{X} \rightarrow F$ et $f_2 : \mathcal{X} \rightarrow F$ deux applications répondant au théorème. Nous introduisons le prédicat $\mathcal{P}(x) : \ll f_1(x) = f_2(x) \gg$. Soit r une règle, donnons nous $(x_1, \dots, x_{n_r}) \in \mathcal{X}^{n_r} \cap \text{Dom}(r)$. Supposons $\mathcal{P}(x_1), \dots, \mathcal{P}(x_{n_r})$. Alors

$$\begin{aligned} f_1(r(x_1, \dots, x_{n_r})) &= g_r(x_1, \dots, x_{n_r}, f_1(x_1), \dots, f_1(x_{n_r})) \\ &= g_r(x_1, \dots, x_{n_r}, f_2(x_1), \dots, f_2(x_{n_r})) \\ &= f_2(r(x_1, \dots, x_{n_r})) \end{aligned}$$

Mais alors on a $\mathcal{P}(r(x_1, \dots, x_{n_r}))$. D'après le théorème de preuve par induction structurelle $\mathcal{P}(x)$ est toujours vrai sur $\mathcal{X}$. Ainsi les applications f_1 et f_2 sont égales. □

Exemple

La définition inductive des entiers $\mathbb{N}$ donnée par

$$\frac{}{0 \in \mathbb{N}} \text{ (zéro)}$$

$$\frac{n \in \mathbb{N}}{n + 1 \in \mathbb{N}} \text{ (succ.)}$$

est non ambiguë.

Il est licite de définir l'application **factorielle** par

Factorielle

Exemple

La définition inductive des entiers $\mathbb{N}$ donnée par

$$\overline{0 \in \mathbb{N}} \text{ (zéro)}$$

$$\frac{n \in \mathbb{N}}{n + 1 \in \mathbb{N}} \text{ (succ.)}$$

est non ambiguë.

Il est licite de définir l'application **factorielle** par

$$\overline{0! = 1} \text{ (zéro)}$$

$$\frac{n \in \mathbb{N}}{(n + 1)! = (n + 1) \times (n!)} \text{ (succ.)}$$

```
def fact(n):  
    if n==0:  
        return 1  
    else:  
        return n * fact(n-1)
```

Longueur d'une liste

Exemple

La définition inductive des listes $\mathcal{L}$ à valeur dans E donnée par

$$\frac{}{\perp \in \mathcal{L}} \text{ (Liste vide)}$$

$$\frac{q \in \mathcal{L}}{(e :: q) \in \mathcal{L}} \text{ (Liste de tête } e)$$

est non ambiguë.

Il est licite de définir l'application **longueur de liste** par

Longueur d'une liste

Exemple

La définition inductive des listes $\mathcal{L}$ à valeur dans E donnée par

$$\frac{}{\perp \in \mathcal{L}} \text{ (Liste vide)} \qquad \frac{q \in \mathcal{L}}{(e :: q) \in \mathcal{L}} \text{ (Liste de tête } e)$$

est non ambiguë.

Il est licite de définir l'application **longueur de liste** par

$$\frac{}{\text{Long}(\perp) = 0} \qquad \frac{q \in \mathcal{L}}{\text{Long}((e :: q)) = \text{Long}(q) + 1}$$

```
class Node:
    def __init__(self, dt, nxt
        ↪ = None):
        self.data = dt
        self.next = nxt
```

```
def length(l):
    if l is None:
        return(0)
    else:
        return(1+length(l.next))
```


Concaténation à une liste

Exemple

Soit ℓ_2 une liste que l'on suppose fixée, il est possible de définir la fonction **concaténation à la liste ℓ_2** par

$$\frac{}{\text{concat}_{\ell_2}(\perp) = \ell_2} \quad \frac{q \in \mathcal{L}}{\text{concat}_{\ell_2}((e :: q)) = (e :: \text{concat}_{\ell_2}(q))}$$

```
def concat(l1, l2):  
    if l1 is None:  
        return(l2)  
    else:  
        return(Node(l1.data, concat(l1.next, l2)))
```

Hauteur d'un arbre binaire

Exemple

La définition inductive arbres binaire donnée par

$$\frac{}{\perp \in \mathcal{A}} \text{ (vide)} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(g|d) \in \mathcal{A}} \text{ (cons.)}$$

est non ambiguë.

Il est licite de définir l'application **hauteur** par

Hauteur d'un arbre binaire

Exemple

La définition inductive arbres binaire donnée par

$$\frac{}{\perp \in \mathcal{A}} \text{ (vide)} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(g|d) \in \mathcal{A}} \text{ (cons.)}$$

est non ambiguë.

Il est licite de définir l'application **hauteur** par

$$\frac{}{\text{haut}(\perp) = -1} \text{ (vide)}$$

Hauteur d'un arbre binaire

Exemple

La définition inductive arbres binaire donnée par

$$\frac{}{\perp \in \mathcal{A}} \text{ (vide)} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(g|d) \in \mathcal{A}} \text{ (cons.)}$$

est non ambiguë.

Il est licite de définir l'application **hauteur** par

$$\frac{}{\text{haut}(\perp) = -1} \text{ (vide)} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{\text{haut}((e|g|d)) = 1 + \max_{\mathbb{Z}}(\text{haut}(g), \text{haut}(d))} \text{ (cons.)}$$

Remarque

La profondeur d'un sommet, qui est le nombre d'ancêtres stricts, ne peut pas être définie inductivement (la profondeur d'une feuille n'est pas constante).

Hauteur d'un arbre : code

```
class Node():
    def __init__(self, d, a = None, b = None):
        self.data = d
        self.left = a
        self.right = b

def height(a):
    if a is None:
        return(-1)
    else:
        return(1 + max(height(a.left), height(a.right)))
```

Maximum d'un arbre étiqueté

Exemple

La définition inductive des arbres binaire étiquetés dans E donnée par

$$\frac{}{\perp \in \mathcal{A}} \text{ (vide)} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(e|g|d) \in \mathcal{A}} \text{ (Arbre de racine } e \in E)$$

est non ambiguë. On suppose que E possède un ordre total.

Il est licite de définir l'application **maximum** par

Maximum d'un arbre étiqueté

Exemple

La définition inductive des arbres binaire étiquetés dans E donnée par

$$\frac{}{\perp \in \mathcal{A}} \text{ (vide)} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(e|g|d) \in \mathcal{A}} \text{ (Arbre de racine } e \in E)$$

est non ambiguë. On suppose que E possède un ordre total.

Il est licite de définir l'application **maximum** par

$$\frac{}{\max_{\mathcal{A}}(\perp) = -\infty} \text{ (vide)} \\ \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{\max_{\mathcal{A}}((e|g|d)) = \max_E(e, \max_{\mathcal{A}}(g), \max_{\mathcal{A}}(d))} \text{ (Arbre de racine } e \in E)$$

Hauteur d'un arbre : code

```
class Node():
    def __init__(self, d, a = None, b = None):
        self.data = d
        self.left = a
        self.right = b

def tree_max(a):
    if a is None:
        return(-sys.maxsize)
    else:
        return(max(a.data, tree_max(a.left), tree_max(a.right)))
```


Évaluation d'expressions arithmétiques : principe

Exemple

La définition inductive des expressions arithmétiques donnée par

$$\frac{}{n \in \mathcal{E}} \text{ (Entier } n)$$

$$\frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{(y+z) \in \mathcal{E}} \text{ (Somme)}$$

$$\frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{(y*z) \in \mathcal{E}} \text{ (Produit)}$$

est non ambiguë (grâce au parenthésage).

Il est licite de définir l'application **évaluation** par

Évaluation d'expressions arithmétiques : principe

Exemple

La définition inductive des expressions arithmétiques donnée par

$$\frac{}{n \in \mathcal{E}} \text{ (Entier } n)$$

$$\frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{(y+z) \in \mathcal{E}} \text{ (Somme)}$$

$$\frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{(y*z) \in \mathcal{E}} \text{ (Produit)}$$

est non ambiguë (grâce au parenthésage).

Il est licite de définir l'application **évaluation** par

$$\frac{}{\text{eval}(n) = n} \text{ (Entier } n)$$

$$\frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{\text{eval}((y+z)) = \text{eval}(y) +_{\mathbb{N}} \text{eval}(z)}$$

$$\frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{\text{eval}((y*z)) = \text{eval}(y) \times_{\mathbb{N}} \text{eval}(z)}$$

Évaluation d'expressions arithmétiques : code

```
class BinaryOperator():
    def __init__(self, a, b):
        self.operand1 = a
        self.operand2 = b
class Sum(BinaryOperator):
    pass
class Product(BinaryOperator):
    pass

def eval(x):
    match x:
        case int() | float():
            return x
        case Sum():
            return(eval(x.operand1) + eval(x.operand2))
        case Product():
            return(eval(x.operand1) * eval(x.operand2))
```

Définition

L'ensemble des valeurs de vérité est un ensemble à deux éléments que l'on note

$$\mathbb{B} = \{\text{Vrai}, \text{Faux}\} = \{\text{True}, \text{False}\} = \{V, F\}.$$

Définition

L'ensemble des **valeurs de vérité** est un ensemble à deux éléments que l'on note

$$\mathbb{B} = \{\text{Vrai}, \text{Faux}\} = \{\mathbf{True}, \mathbf{False}\} = \{V, F\}.$$

Définition

Une **valuation** de l'ensemble des variables booléennes $\mathcal{V} = \{x_1, x_2, \dots\}$ est une distribution des valeurs de vérité sur l'ensemble des variables propositionnelles $\mathcal{V}$, autrement dit une fonction

$$\alpha : \mathcal{V} \rightarrow \mathbb{B}.$$

Table de vérité des connecteurs

Nous définissons exhaustivement trois fonctions booléennes $f_{\neg} : \mathbb{B} \rightarrow \mathbb{B}$, $f_{\vee} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$, $f_{\wedge} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$ par les tables :

Table de vérité des connecteurs

Nous définissons exhaustivement trois fonctions booléennes $f_{\neg} : \mathbb{B} \rightarrow \mathbb{B}$, $f_{\vee} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$, $f_{\wedge} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$ par les tables :

x	$f_{\neg}(x)$
V	F
F	V

Table de vérité des connecteurs

Nous définissons exhaustivement trois fonctions booléennes

$f_{\neg} : \mathbb{B} \rightarrow \mathbb{B}$, $f_{\vee} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$, $f_{\wedge} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$ par les tables :

x	$f_{\neg}(x)$
V	F
F	V

x	y	$f_{\vee}(x, y)$
V	V	V
V	F	V
F	V	V
F	F	F

Table de vérité des connecteurs

Nous définissons exhaustivement trois fonctions booléennes $f_{\neg} : \mathbb{B} \rightarrow \mathbb{B}$, $f_{\vee} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$, $f_{\wedge} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$ par les tables :

x	$f_{\neg}(x)$
V	F
F	V

x	y	$f_{\vee}(x, y)$
V	V	V
V	F	V
F	V	V
F	F	F

x	y	$f_{\wedge}(x, y)$
V	V	V
V	F	F
F	V	F
F	F	F

Évaluation d'une formule de logique

L'évaluation d'une formule logique φ d'après la valuation α est la valeur de vérité, notée $\llbracket \varphi \rrbracket_\alpha$, définie inductivement par

Évaluation d'une formule de logique

L'évaluation d'une formule logique φ d'après la valuation α est la valeur de vérité, notée $\llbracket \varphi \rrbracket_\alpha$, définie inductivement par

$$\begin{array}{c} \frac{}{\llbracket x_n \rrbracket_\alpha = \alpha(x_n)} \text{ (Variable } n) \\[1em] \frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{\llbracket \varphi \vee \psi \rrbracket_\alpha = f_\vee(\llbracket \varphi \rrbracket_\alpha, \llbracket \psi \rrbracket_\alpha)} \text{ (Ou)} \\[1em] \frac{\varphi \in \mathcal{F}}{\llbracket \neg \varphi \rrbracket_\alpha = f_\neg(\llbracket \varphi \rrbracket_\alpha)} \text{ (Negation)} \\[1em] \frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{\llbracket \varphi \wedge \psi \rrbracket_\alpha = f_\wedge(\llbracket \varphi \rrbracket_\alpha, \llbracket \psi \rrbracket_\alpha)} \text{ (Et)} \end{array}$$

Évaluation de formules de logique : code

```
class Neg():
    def __init__(self, a):
        self.operand = a
class BinaryOperator():
    def __init__(self, a, b):
        self.operand1 = a
        self.operand2 = b
class Or(BinaryOperator):
    pass
class And(BinaryOperator):
    pass

def eval(x, alpha):
    match x:
        case int():
            return alpha[x]
        case Neg():
            return(not eval(x.operand, alpha))
        case Or():
            return(eval(x.operand1, alpha) or eval(x.operand2, alpha))
        case And():
            return(eval(x.operand1, alpha) and eval(x.operand2, alpha))
```

Parcours d'un arbre étiqueté dans l'ordre préfixe

Exemple

La définition inductive des arbres binaires étiquetés dans E donnée par

$$\frac{}{\perp \in \mathcal{A}} \text{ (vide)} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(e|g|d) \in \mathcal{A}} \text{ (Arbre de racine } e \in E)$$

est non ambiguë.

Il est licite de définir l'application **parcours préfixe** par

Parcours d'un arbre étiqueté dans l'ordre préfixe

Exemple

La définition inductive des arbres binaires étiquetés dans E donnée par

$$\frac{}{\perp \in \mathcal{A}} \text{ (vide)} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(e|g|d) \in \mathcal{A}} \text{ (Arbre de racine } e \in E)$$

est non ambiguë.

Il est licite de définir l'application **parcours préfixe** par

$$\frac{}{\text{préf}(\perp) = \text{rien}} \text{ (vide)} \\ \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{\text{préf}((e|g|d)) = \begin{array}{|l} \text{examen}(e) \\ \text{préf}(g) \\ \text{préf}(d) \end{array}} \text{ (Arbre de racine } e \in E)$$

Parcours d'un arbre étiqueté dans l'ordre préfixe et affichage

```
class Node():
    def __init__(self, d, a = None, b = None):
        self.data = d
        self.left = a
        self.right = b

def prefix_traversal(a, visit):
    if isinstance(a, Node):
        visit(a)
        prefix_traversal(a.left, visit)
        prefix_traversal(a.right, visit)

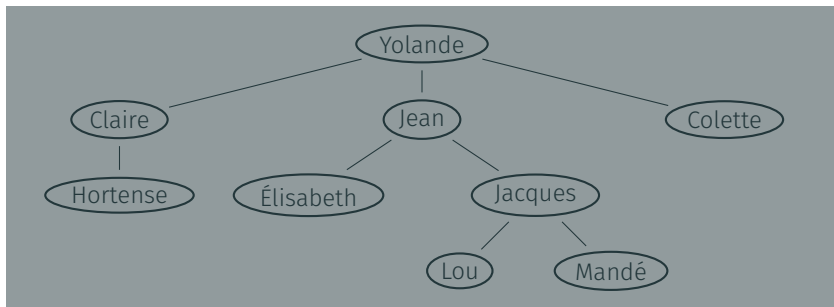
def root_print(a):
    print(a.data)

# On suppose arbre défini
prefix_traversal(arbre, root_print)
```

Parcours préfixe illustré

On peut encore parler de parcours préfixe dans le cas d'arbres généraux.

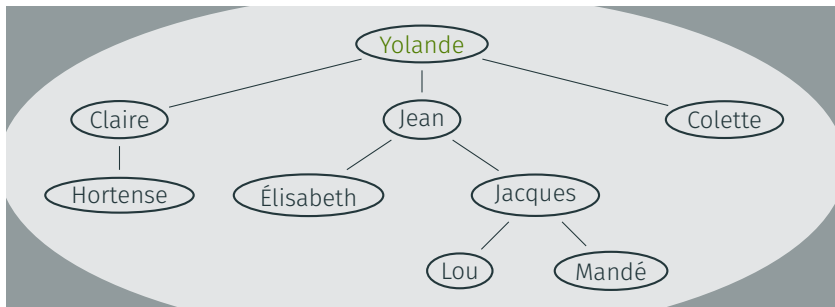
Exemple



Parcours préfixe illustré

On peut encore parler de parcours préfixe dans le cas d'arbres généraux.

Exemple

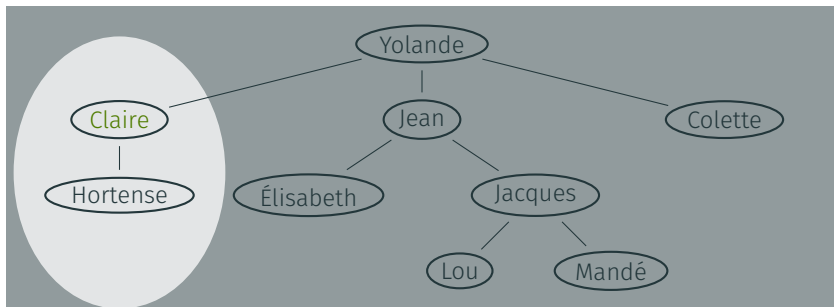


Yolande

Parcours préfixe illustré

On peut encore parler de parcours préfixe dans le cas d'arbres généraux.

Exemple

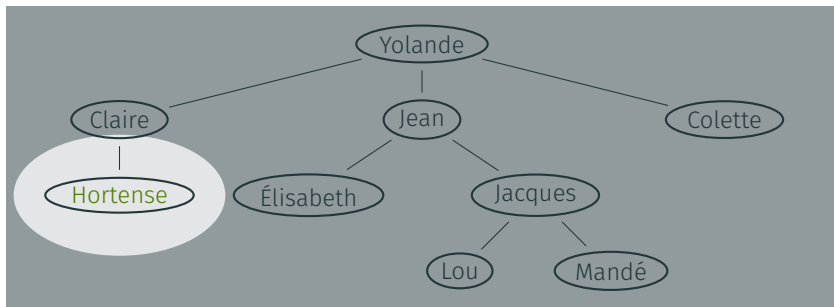


Yolande, Claire

Parcours préfixe illustré

On peut encore parler de parcours préfixe dans le cas d'arbres généraux.

Exemple

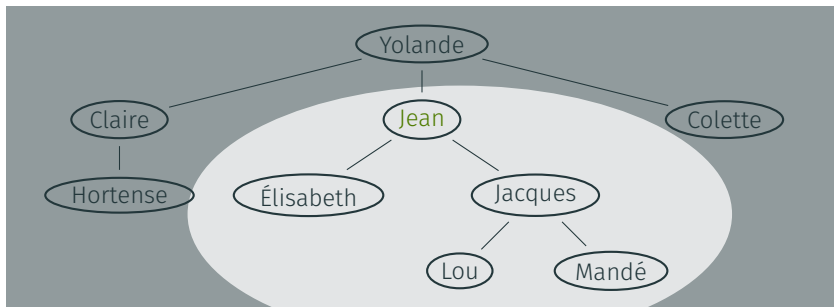


Yolande, Claire, Hortense

Parcours préfixe illustré

On peut encore parler de parcours préfixe dans le cas d'arbres généraux.

Exemple

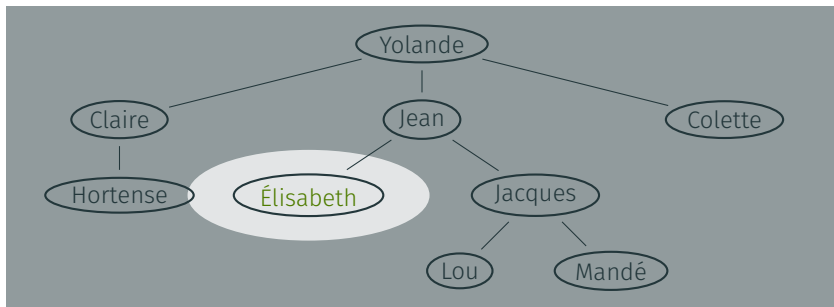


Yolande, Claire, Hortense, Jean

Parcours préfixe illustré

On peut encore parler de parcours préfixe dans le cas d'arbres généraux.

Exemple

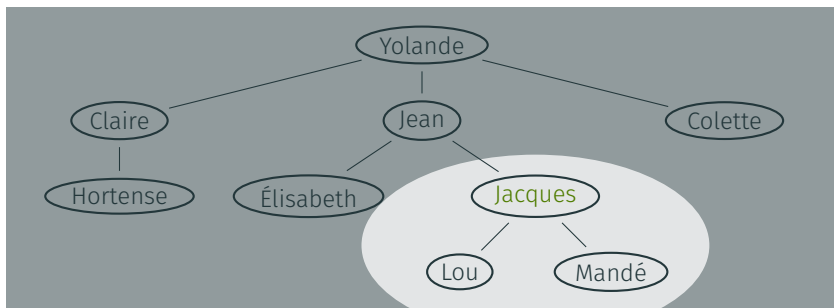


Yolande, Claire, Hortense, Jean, Élisabeth

Parcours préfixe illustré

On peut encore parler de parcours préfixe dans le cas d'arbres généraux.

Exemple

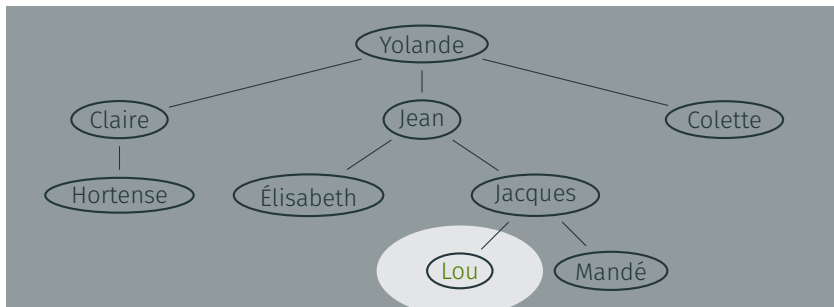


Yolande, Claire, Hortense, Jean, Élisabeth, Jacques

Parcours préfixe illustré

On peut encore parler de parcours préfixe dans le cas d'arbres généraux.

Exemple

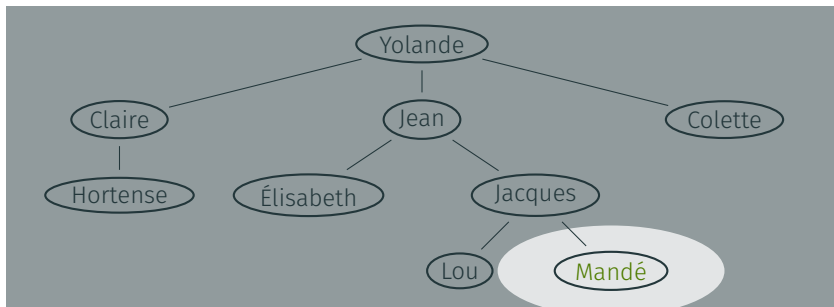


Yolande, Claire, Hortense, Jean, Élisabeth, Jacques, Lou

Parcours préfixe illustré

On peut encore parler de parcours préfixe dans le cas d'arbres généraux.

Exemple

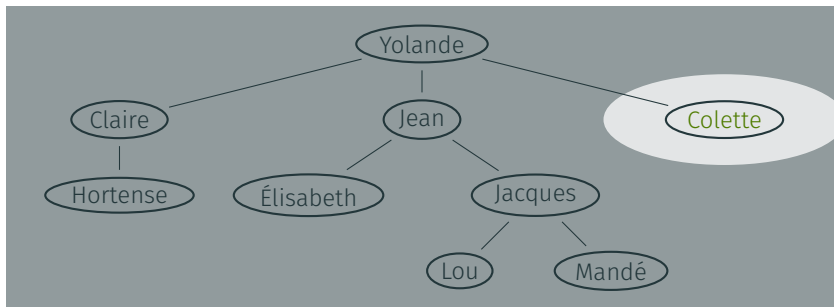


Yolande, Claire, Hortense, Jean, Élisabeth, Jacques, Lou, Mandé

Parcours préfixe illustré

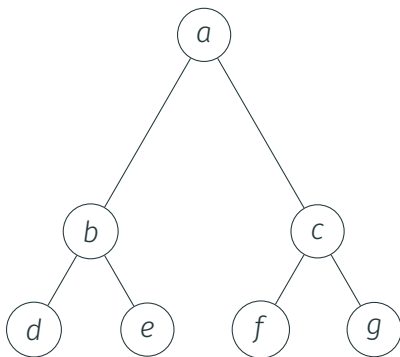
On peut encore parler de parcours préfixe dans le cas d'arbres généraux.

Exemple



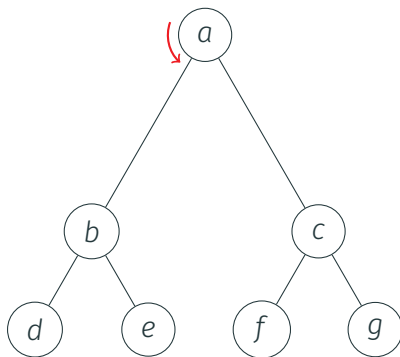
Yolande, Claire, Hortense, Jean, Élisabeth, Jacques, Lou, Mandé, Colette.

Préordre et parcours à main gauche



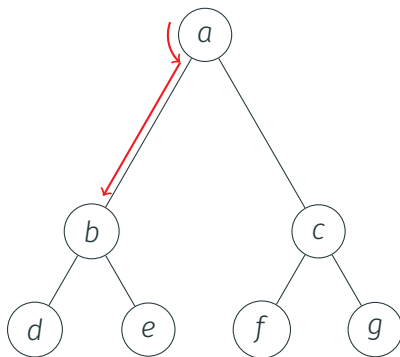
Parcours préfixe (première rencontre) :

Préordre et parcours à main gauche



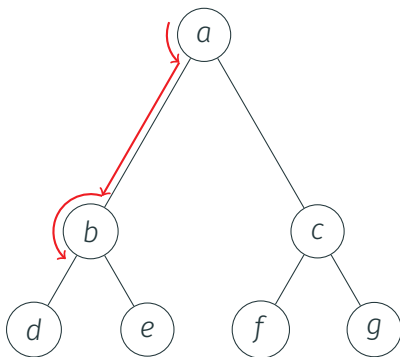
Parcours préfixe (première rencontre) : *a*,

Préordre et parcours à main gauche



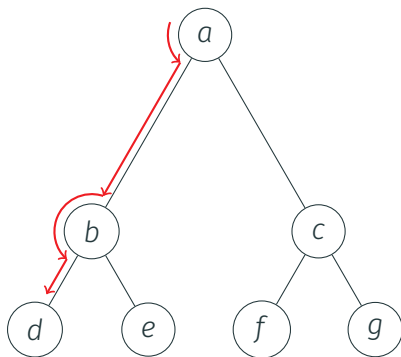
Parcours préfixe (première rencontre) : *a*,

Préordre et parcours à main gauche



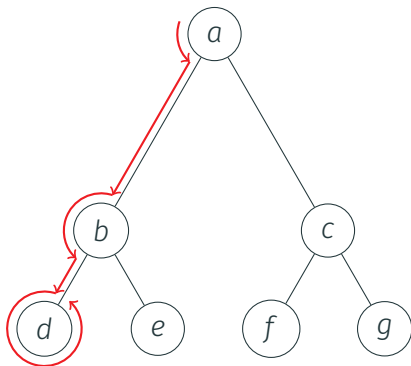
Parcours préfixe (première rencontre) : *a, b,*

Préordre et parcours à main gauche



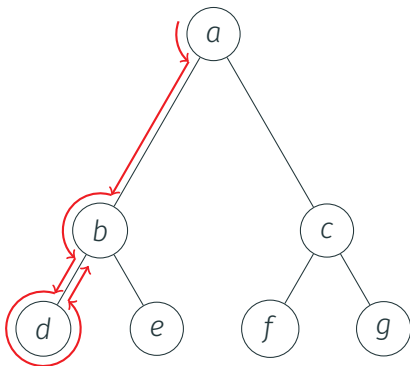
Parcours préfixe (première rencontre) : *a, b,*

Préordre et parcours à main gauche



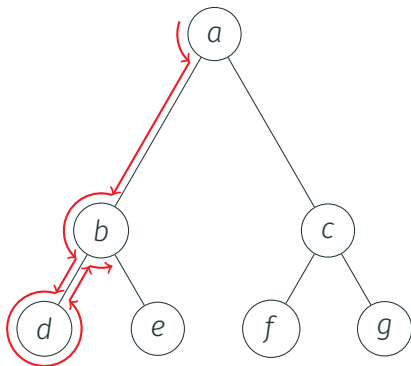
Parcours préfixe (première rencontre) : $a, b, d,$

Préordre et parcours à main gauche



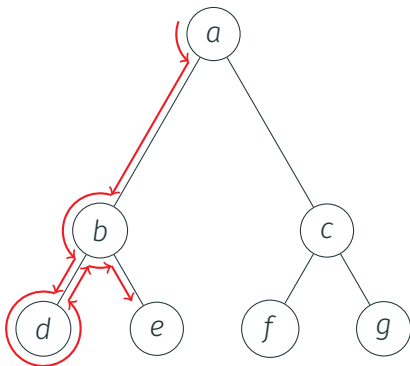
Parcours préfixe (première rencontre) : *a*, *b*, *d*,

Préordre et parcours à main gauche



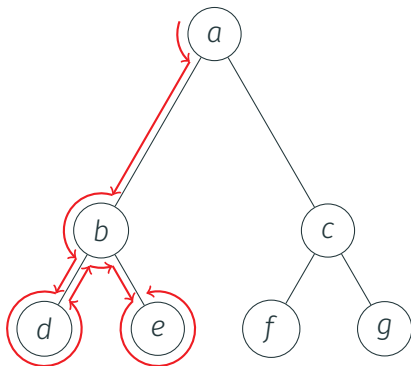
Parcours préfixe (première rencontre) : *a*, *b*, *d*,

Préordre et parcours à main gauche



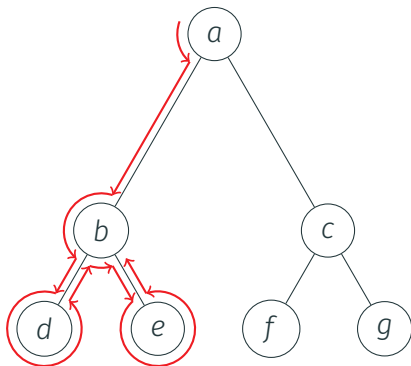
Parcours préfixe (première rencontre) : *a*, *b*, *d*,

Préordre et parcours à main gauche



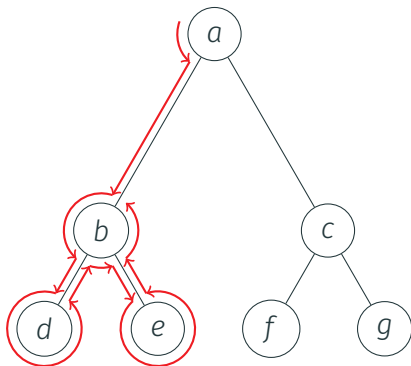
Parcours préfixe (première rencontre) : *a*, *b*, *d*, *e*,

Préordre et parcours à main gauche



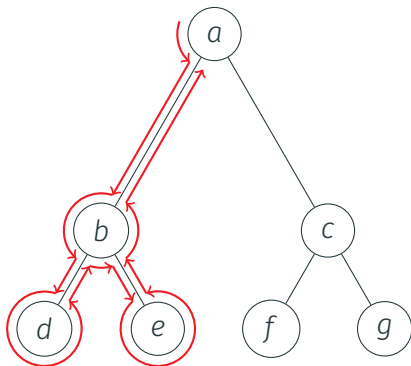
Parcours préfixe (première rencontre) : $a, b, d, e,$

Préordre et parcours à main gauche



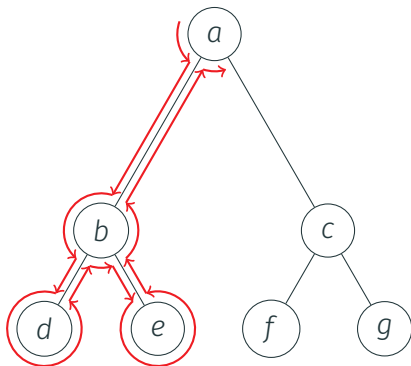
Parcours préfixe (première rencontre) : $a, b, d, e,$

Préordre et parcours à main gauche



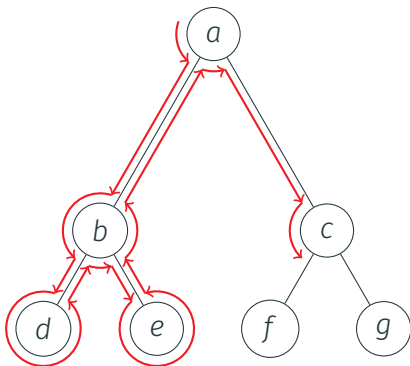
Parcours préfixe (première rencontre) : *a*, *b*, *d*, *e*,

Préordre et parcours à main gauche



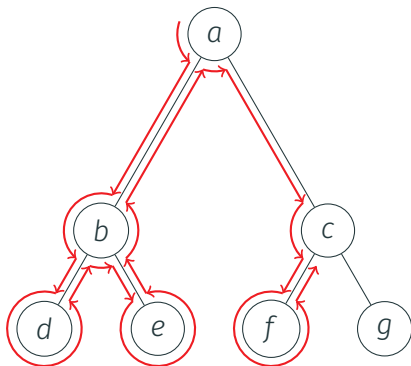
Parcours préfixe (première rencontre) : *a*, *b*, *d*, *e*,

Préordre et parcours à main gauche



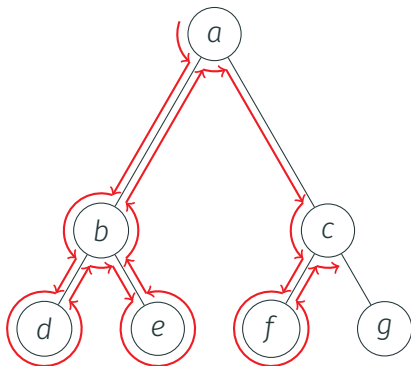
Parcours préfixe (première rencontre) : $a, b, d, e, c,$

Préordre et parcours à main gauche



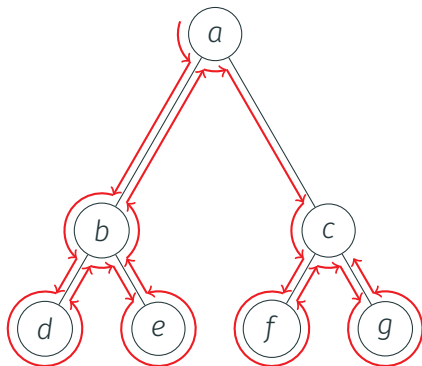
Parcours préfixe (première rencontre) : $a, b, d, e, c, f,$

Préordre et parcours à main gauche



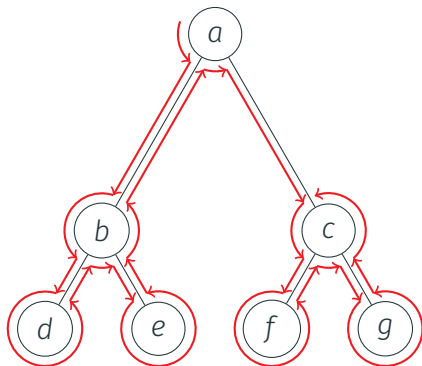
Parcours préfixe (première rencontre) : *a*, *b*, *d*, *e*, *c*, *f*,

Préordre et parcours à main gauche



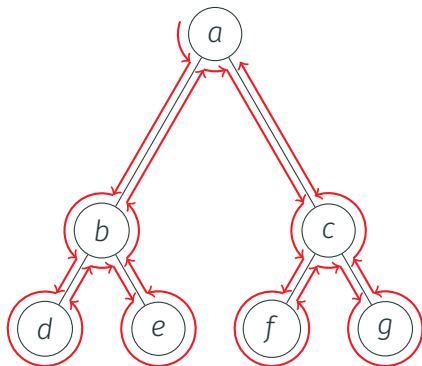
Parcours préfixe (première rencontre) : *a*, *b*, *d*, *e*, *c*, *f*, *g*.

Préordre et parcours à main gauche



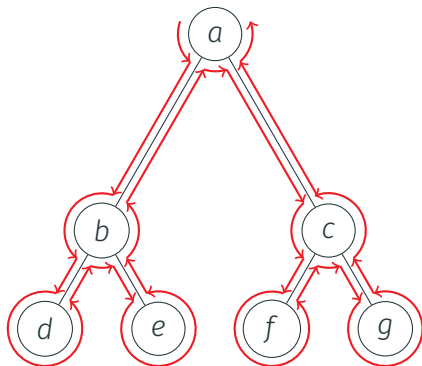
Parcours préfixe (première rencontre) : *a, b, d, e, c, f, g*.

Préordre et parcours à main gauche



Parcours préfixe (première rencontre) : a, b, d, e, c, f, g .

Préordre et parcours à main gauche



Parcours préfixe (première rencontre) : *a, b, d, e, c, f, g*.

Parcours d'un arbre étiqueté dans l'ordre infixe

Exemple

La définition inductive arbres binaire étiquetés dans E donnée par

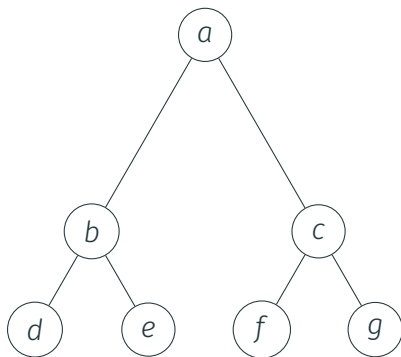
$$\frac{}{\perp \in \mathcal{A}} \text{ (vide)} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(e|g|d) \in \mathcal{A}} \text{ (Arbre de racine } e \in E)$$

est non ambiguë.

Il est licite de définir l'application **parcours infixe** par

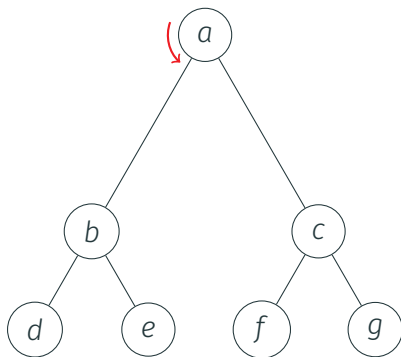
$$\frac{}{\text{infix}(\perp) = \text{rien}} \text{ (vide)}$$
$$\frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{\text{infix}((e|g|d)) = \begin{array}{|l} \text{infix}(g) \\ \text{examen}(e) \\ \text{infix}(d) \end{array}} \text{ (Arbre de racine } e \in E)$$

Ordre infixe et parcours à main gauche



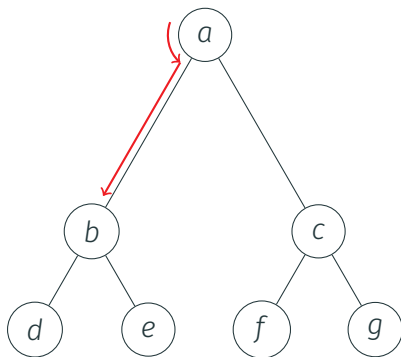
Parcours infixe (passage sous le sommet) :

Ordre infixe et parcours à main gauche



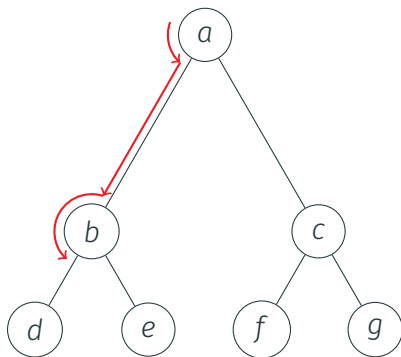
Parcours infixe (passage sous le sommet) :

Ordre infixe et parcours à main gauche



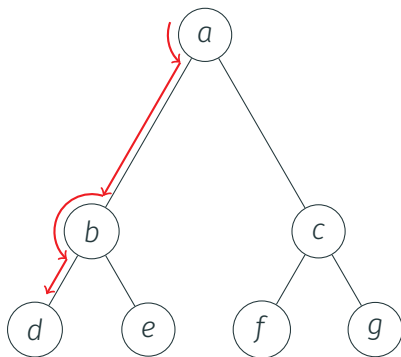
Parcours infixe (passage sous le sommet) :

Ordre infixe et parcours à main gauche



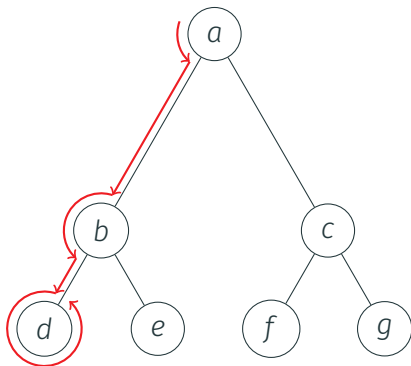
Parcours infixe (passage sous le sommet) :

Ordre infixe et parcours à main gauche



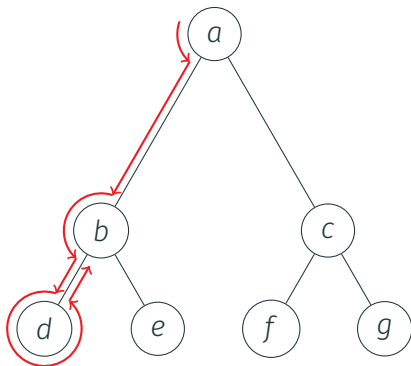
Parcours infixe (passage sous le sommet) :

Ordre infixe et parcours à main gauche



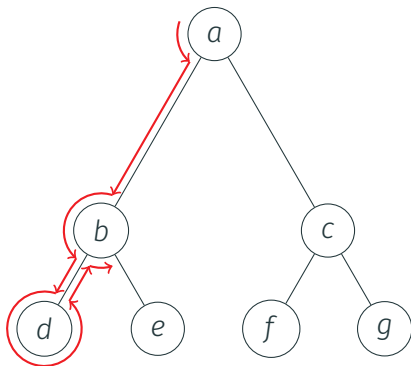
Parcours infixe (passage sous le sommet) : *d*,

Ordre infixe et parcours à main gauche



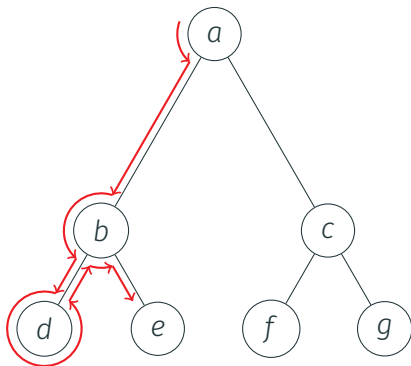
Parcours infixe (passage sous le sommet) : *d*,

Ordre infixe et parcours à main gauche



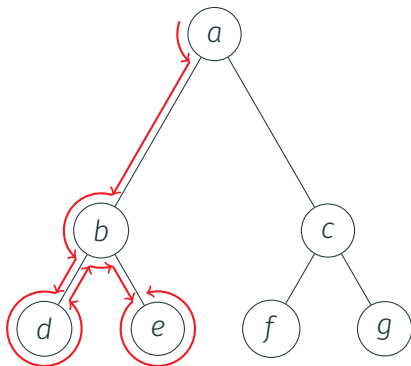
Parcours infixe (passage sous le sommet) : d , b ,

Ordre infixe et parcours à main gauche



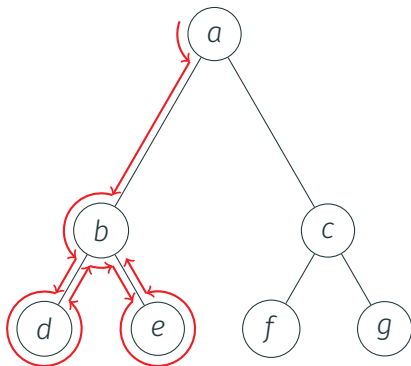
Parcours infixe (passage sous le sommet) : *d*, *b*,

Ordre infixe et parcours à main gauche



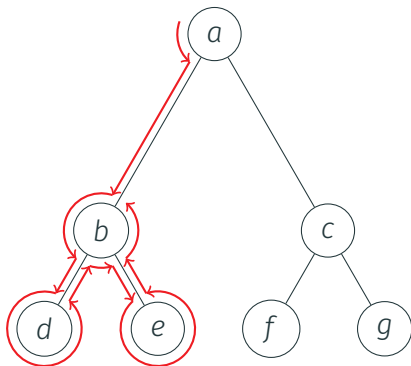
Parcours infixe (passage sous le sommet) : *d, b, e,*

Ordre infixe et parcours à main gauche



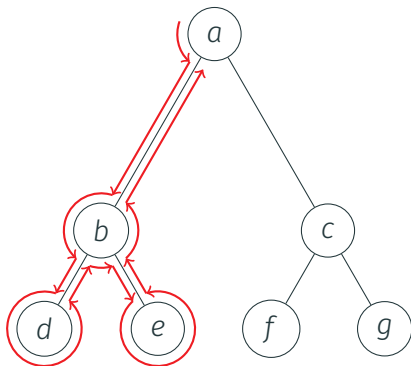
Parcours infixe (passage sous le sommet) : *d*, *b*, *e*,

Ordre infixe et parcours à main gauche



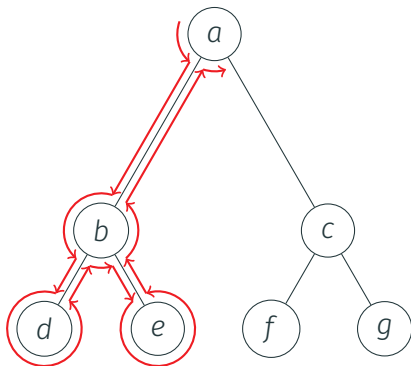
Parcours infixe (passage sous le sommet) : *d, b, e,*

Ordre infixe et parcours à main gauche



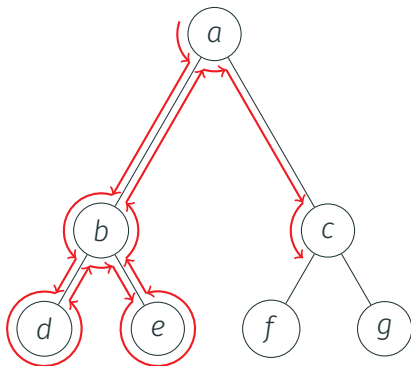
Parcours infixe (passage sous le sommet) : *d, b, e,*

Ordre infixe et parcours à main gauche



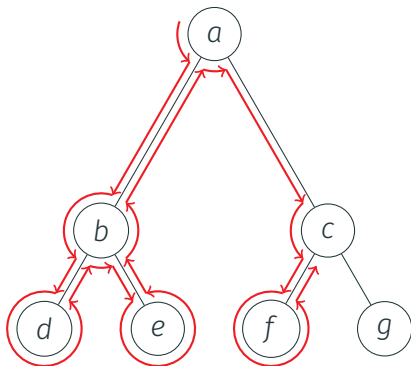
Parcours infixe (passage sous le sommet) : *d, b, e, a,*

Ordre infixe et parcours à main gauche



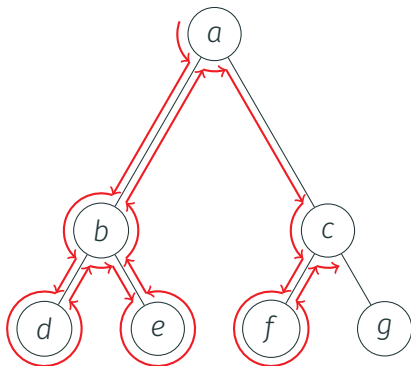
Parcours infixe (passage sous le sommet) : $d, b, e, a,$

Ordre infixe et parcours à main gauche



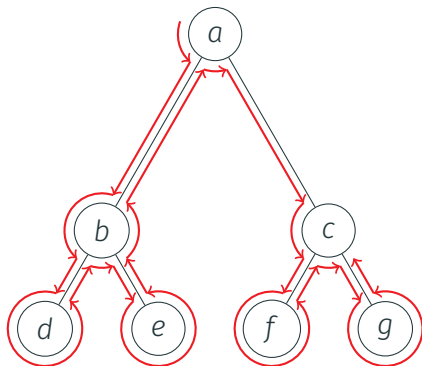
Parcours infixe (passage sous le sommet) : $d, b, e, a, f,$

Ordre infixe et parcours à main gauche



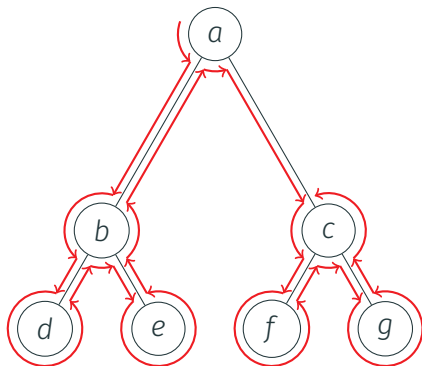
Parcours infixe (passage sous le sommet) : $d, b, e, a, f, c,$

Ordre infixe et parcours à main gauche



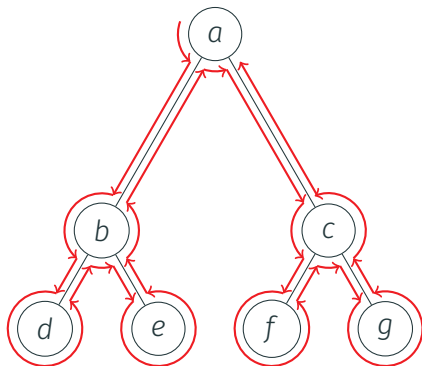
Parcours infixe (passage sous le sommet) : d, b, e, a, f, c, g .

Ordre infixe et parcours à main gauche



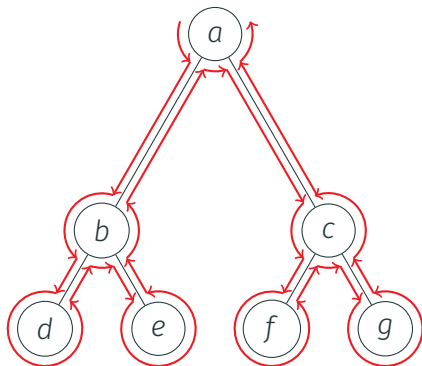
Parcours infixe (passage sous le sommet) : d, b, e, a, f, c, g .

Ordre infixe et parcours à main gauche



Parcours infixe (passage sous le sommet) : d, b, e, a, f, c, g .

Ordre infixe et parcours à main gauche



Parcours infixe (passage sous le sommet) : d, b, e, a, f, c, g .

Parcours d'un arbre étiqueté dans l'ordre suffixe

Exemple

La définition inductive arbres binaire étiquetés dans E donnée par

$$\frac{}{\perp \in \mathcal{A}} \text{ (vide)} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(e|g|d) \in \mathcal{A}} \text{ (Arbre de racine } e \in E)$$

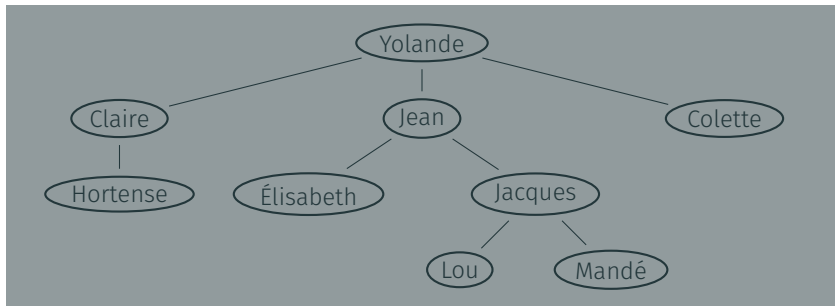
est non ambiguë.

Il est licite de définir l'application **parcours suffixe** par

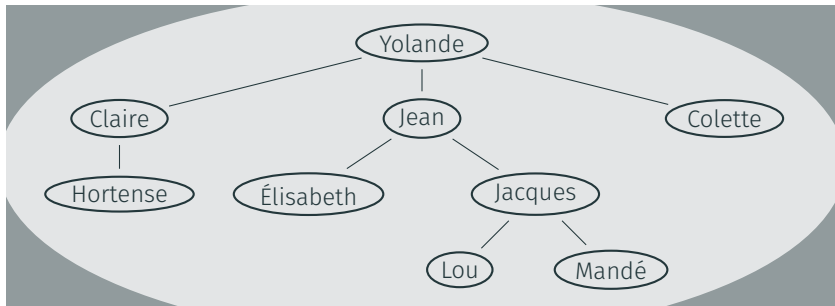
$$\frac{}{\text{postfix}(\perp) = \text{rien}} \text{ (vide)}$$
$$\frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{\text{postfix}((e|g|d)) = \begin{array}{|l} \text{postfix}(g) \\ \text{postfix}(d) \\ \text{examen}(e) \end{array}} \text{ (Arbre de racine } e \in E)$$

Parcours postfixe

Exemple

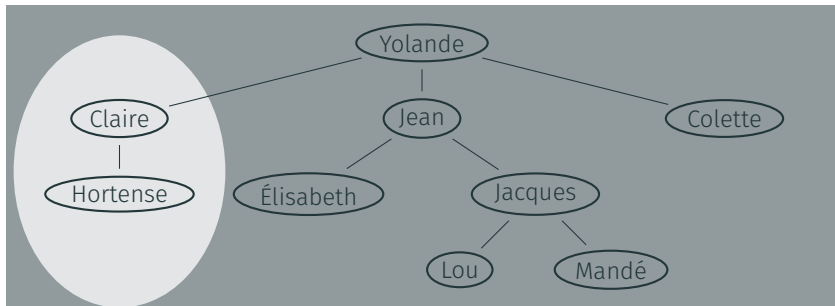


Exemple



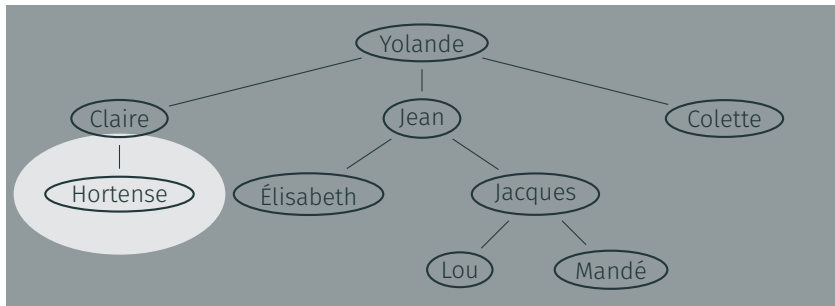
Parcours postfixe

Exemple



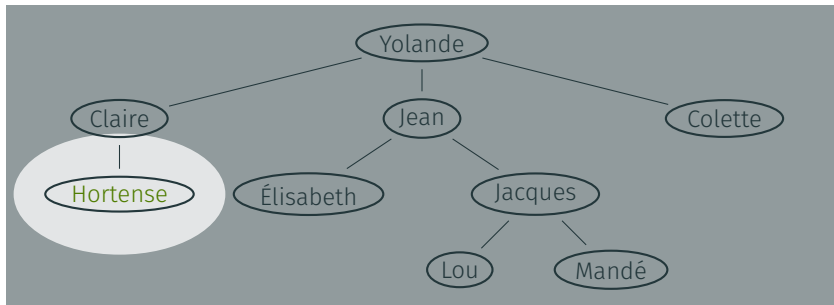
Parcours postfixe

Exemple



Parcours postfixe

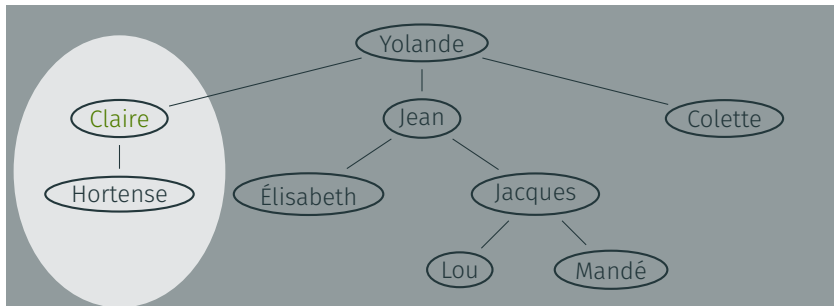
Exemple



Hortense

Parcours postfixe

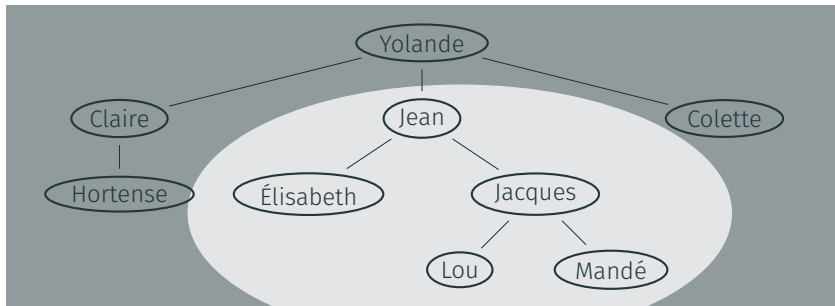
Exemple



Hortense, Claire

Parcours postfixe

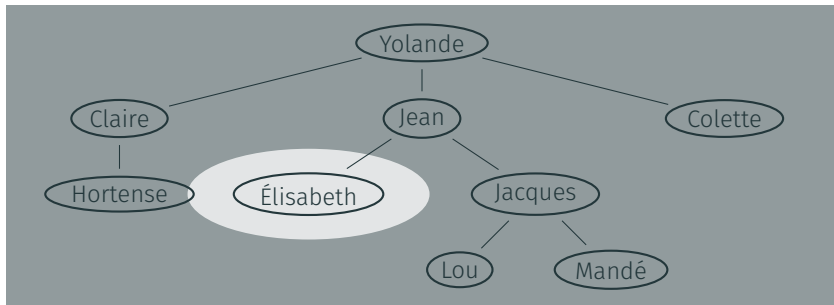
Exemple



Hortense, Claire

Parcours postfixe

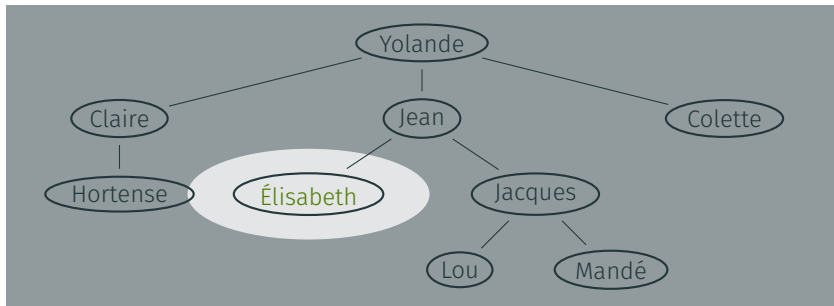
Exemple



Hortense, Claire

Parcours postfixe

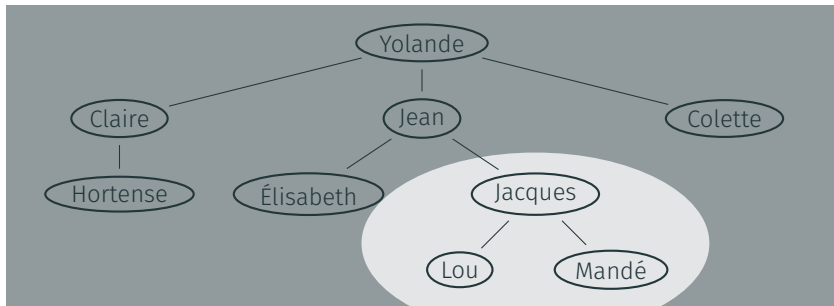
Exemple



Hortense, Claire, Élisabeth

Parcours postfixe

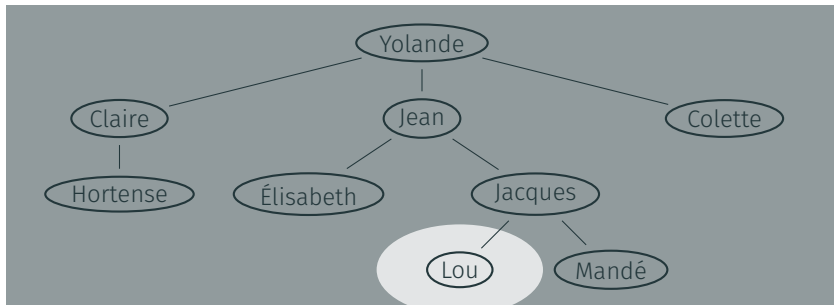
Exemple



Hortense, Claire, Élisabeth

Parcours postfixe

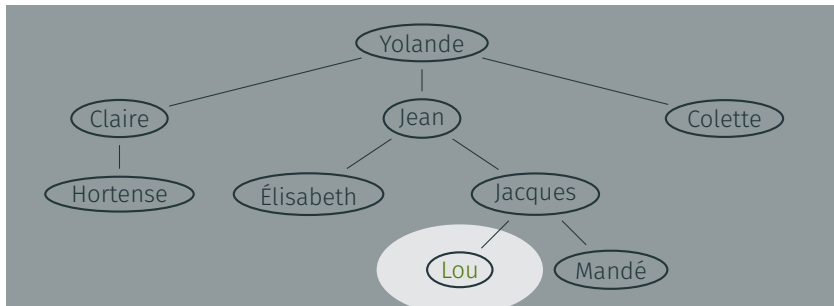
Exemple



Hortense, Claire, Élisabeth

Parcours postfixe

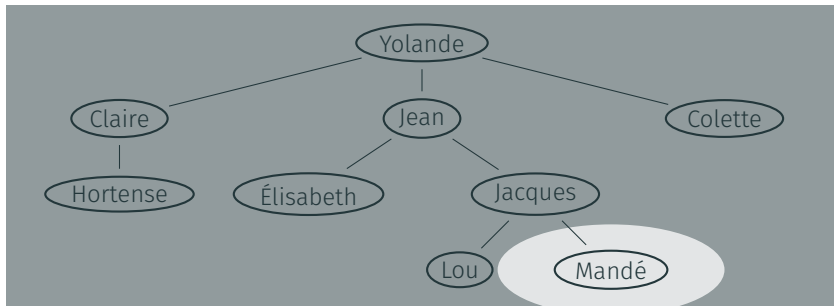
Exemple



Hortense, Claire, Élisabeth, Lou

Parcours postfixe

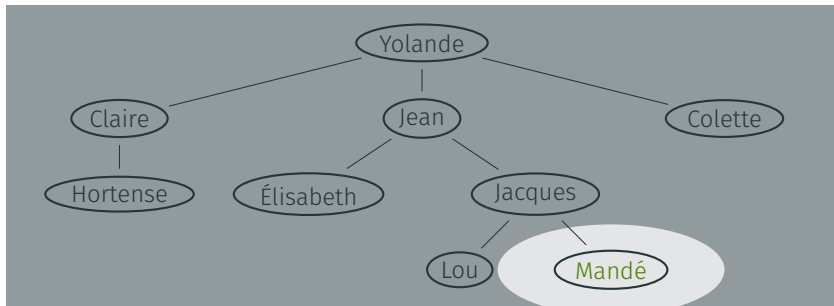
Exemple



Hortense, Claire, Élisabeth, Lou

Parcours postfixe

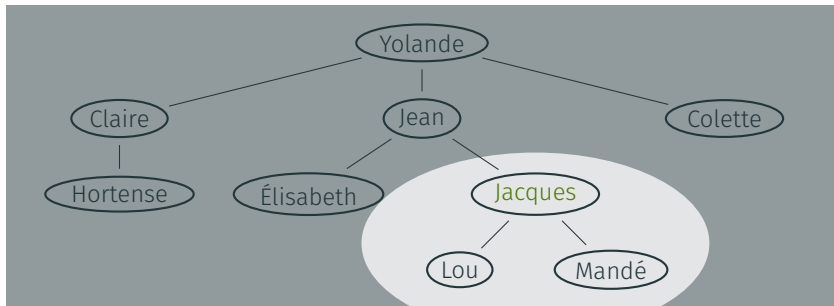
Exemple



Hortense, Claire, Élisabeth, Lou, Mandé

Parcours postfixe

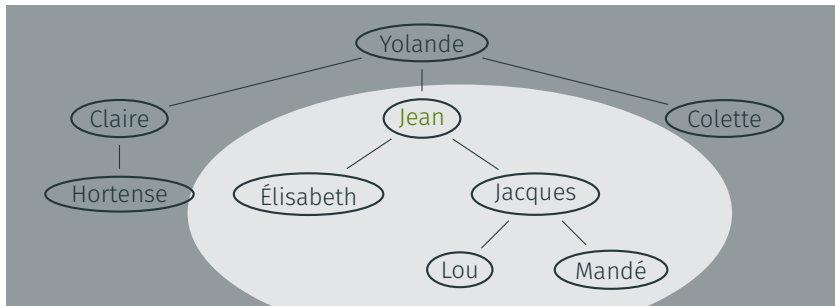
Exemple



Hortense, Claire, Élisabeth, Lou, Mandé, Jacques

Parcours postfixe

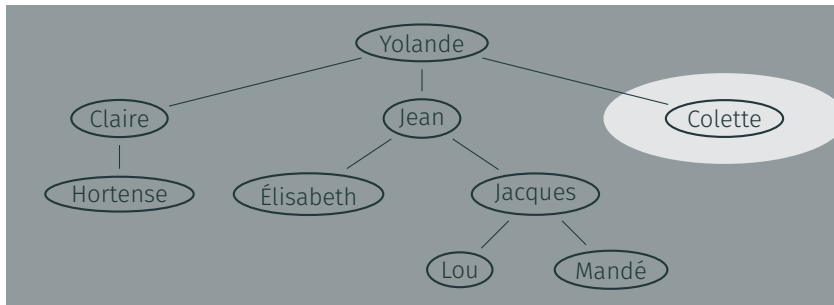
Exemple



Hortense, Claire, Élisabeth, Lou, Mandé, Jacques, Jean

Parcours postfixe

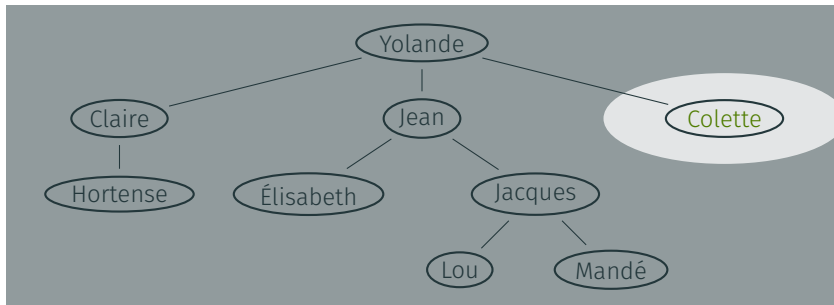
Exemple



Hortense, Claire, Élisabeth, Lou, Mandé, Jacques, Jean

Parcours postfixe

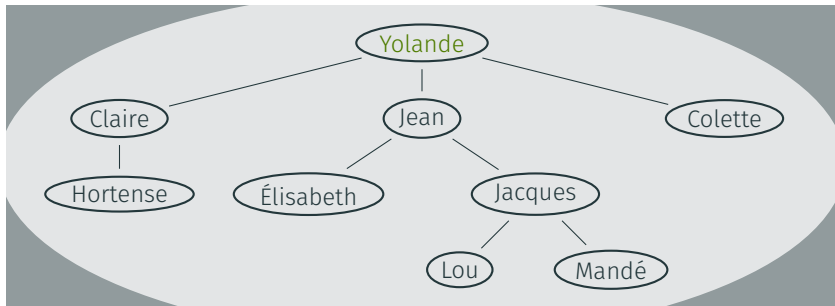
Exemple



Hortense, Claire, Élisabeth, Lou, Mandé, Jacques, Jean, Colette

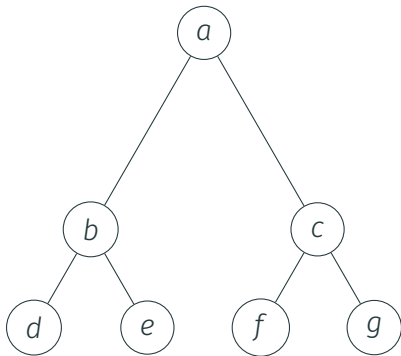
Parcours postfixe

Exemple



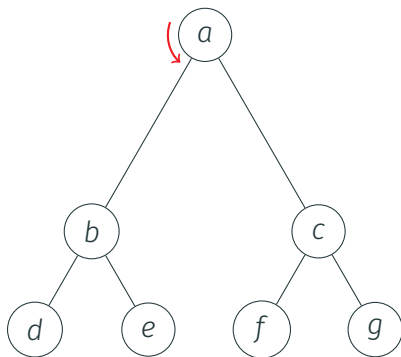
Hortense, Claire, Élisabeth, Lou, Mandé, Jacques, Jean, Colette, Yolande.

Postordre et parcours à main gauche



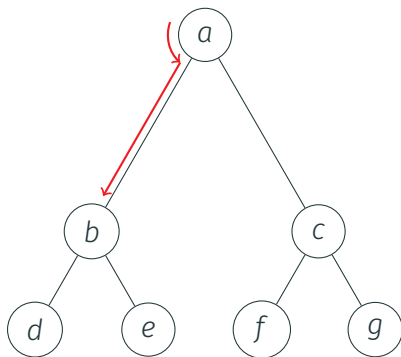
Parcours postfixe (dernière rencontre) :

Postordre et parcours à main gauche



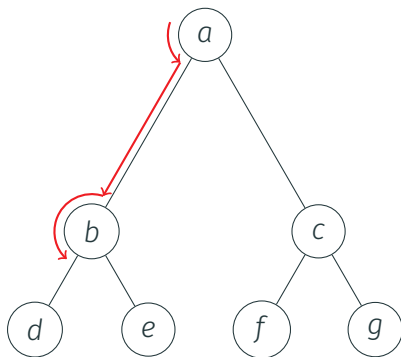
Parcours postfixe (dernière rencontre) :

Postordre et parcours à main gauche



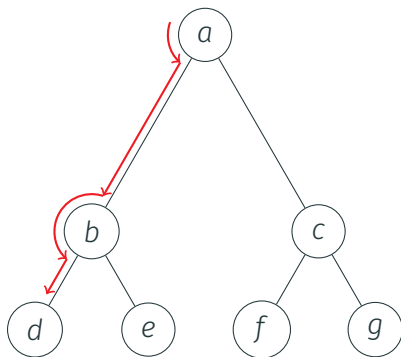
Parcours postfixe (dernière rencontre) :

Postordre et parcours à main gauche



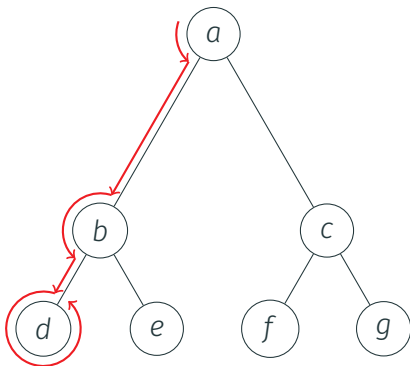
Parcours postfixe (dernière rencontre) :

Postordre et parcours à main gauche



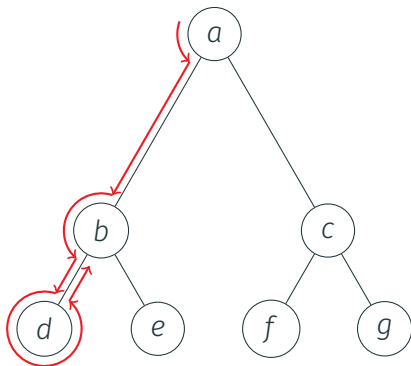
Parcours postfixe (dernière rencontre) :

Postordre et parcours à main gauche



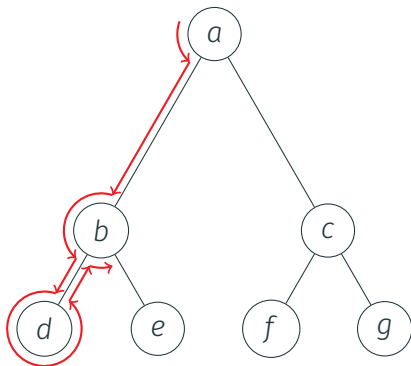
Parcours postfixe (dernière rencontre) : d ,

Postordre et parcours à main gauche



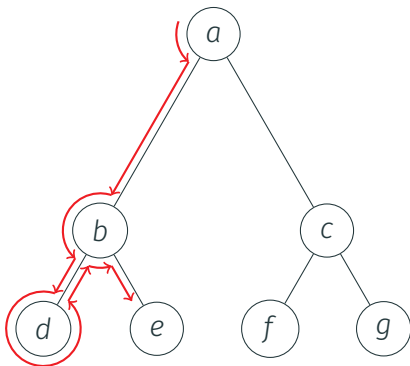
Parcours postfixe (dernière rencontre) : d ,

Postordre et parcours à main gauche



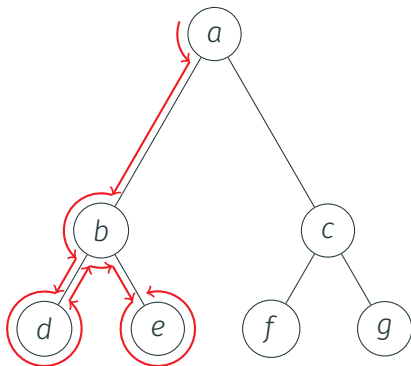
Parcours postfixe (dernière rencontre) : d ,

Postordre et parcours à main gauche



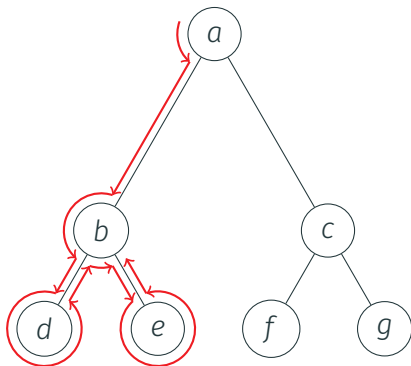
Parcours postfixe (dernière rencontre) : *d*,

Postordre et parcours à main gauche



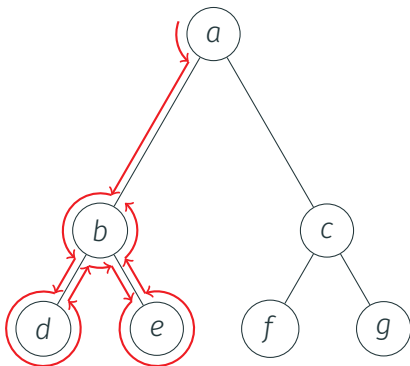
Parcours postfixe (dernière rencontre) : *d*, *e*,

Postordre et parcours à main gauche



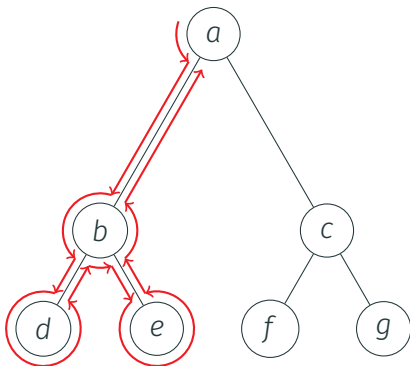
Parcours postfixe (dernière rencontre) : *d*, *e*,

Postordre et parcours à main gauche



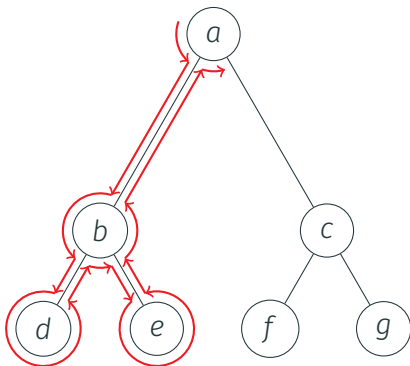
Parcours postfixe (dernière rencontre) : *d*, *e*, *b*,

Postordre et parcours à main gauche



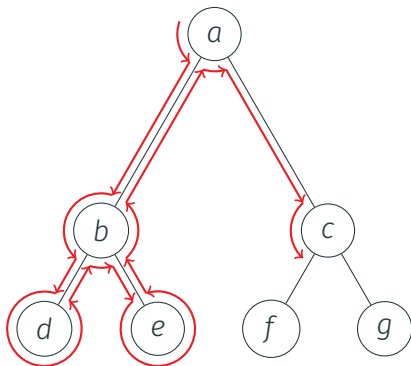
Parcours postfixe (dernière rencontre) : *d*, *e*, *b*,

Postordre et parcours à main gauche



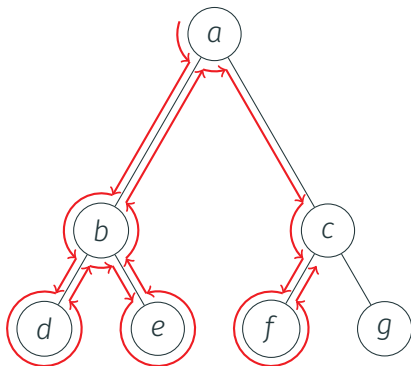
Parcours postfixe (dernière rencontre) : *d*, *e*, *b*,

Postordre et parcours à main gauche



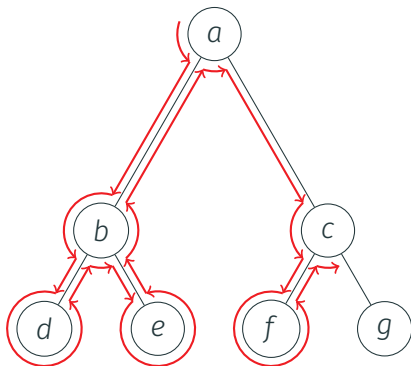
Parcours postfixe (dernière rencontre) : *d*, *e*, *b*,

Postordre et parcours à main gauche



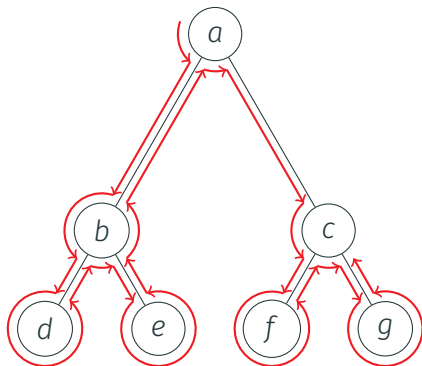
Parcours postfixe (dernière rencontre) : $d, e, b, f,$

Postordre et parcours à main gauche



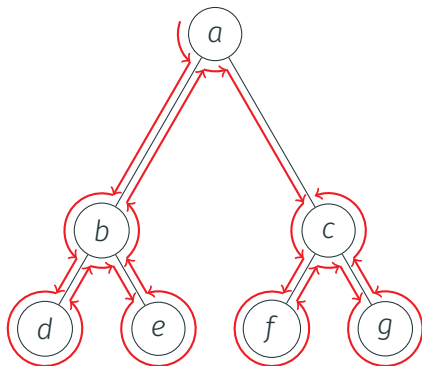
Parcours postfixe (dernière rencontre) : $d, e, b, f,$

Postordre et parcours à main gauche



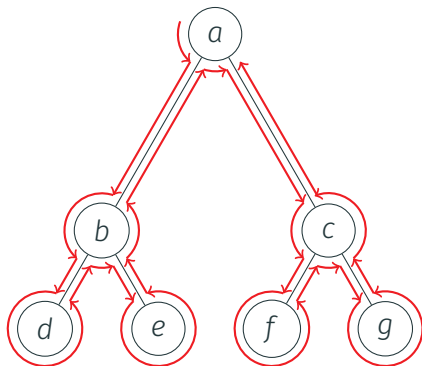
Parcours postfixe (dernière rencontre) : $d, e, b, f, g,$

Postordre et parcours à main gauche



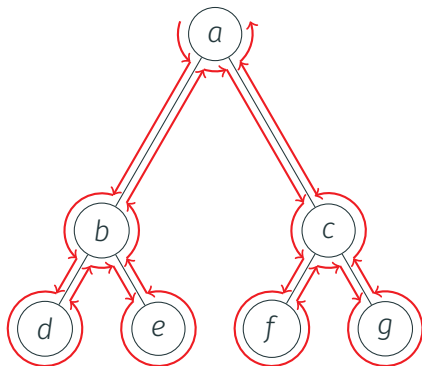
Parcours postfixe (dernière rencontre) : $d, e, b, f, g, c,$

Postordre et parcours à main gauche



Parcours postfixe (dernière rencontre) : $d, e, b, f, g, c,$

Postordre et parcours à main gauche



Parcours postfixe (dernière rencontre) : d, e, b, f, g, c, a .